CS

23COC251

F018482

Event Sharing and Organisational Planning Mobile Application

by

Daniel P. Cuba

Supervisor: Dr Sara Saravi

Department of Computer Science

Loughborough University

May/June 2024

Abstract

With the introduction of social media, the world is the most connected it has been in the existence of humanity. Humans can interact across the internet frivolously, however, making plans and connecting with friends and family, has also come at greater difficulty, with the populus becoming busier.

This mobile application sets out to improve the ability for users to connect in real life by offering events, and allowing users to see who of their friends are attending said event, and whether it fits within their increasingly tight schedule.

Table of Contents

Abstract 2
Table of Contents 3
Table of Tables 7
Table of Figures 8
1. Introduction 13
1.1 Problem Domain 13
1.2 Motivations 13
1.3 Aims and Objectives 14
2. Literature Review 15
2.1 Android vs IOS: 15
2.2 Market Research and Gap Analysis 16
2.2.1 Skiddle 16
2.2.2 EventBrite 17
2.2.3 Conclusion 18
3. Methodology and Requirements 19
3.1 Local Database 19
3.1.1 Model 20
3.1.2 ViewModel 20
3.1.3 View 20
3.2 Online Database 21
3.2.1 Firebase Authentication 21
3.2.2 Cloud Firestore 21
3.2.3 Firebase Storage 22
3.3 Requirements 23
3.3.1 Sign-in Screen 23

3.3.2 Home Screen 25
3.3.3 Public Event View Screen 26
3.3.4 Search Screen 28
3.3.5 Calendar Screen 29
3.3.6 Day View Screen 31
3.3.7 Create Private Events Screen 32
3.3.8 Create Public Events Screen 34
3.3.9 Profile Screen 36
3.3.10 Settings Screen 37
4. Design and Development 39
4.1 Design Approach 39
4.2 Local Database Implementation 40
4.2.1 Model 40
4.2.2 ViewModel 42
4.2.3 View 43
4.3 Online Database 44
4.3.1 Cloud Firestore 44
4.3.2 Firebase Storage 45
4.4 Sign-up/Login Screen 46
4.5 Home Screen 47
4.5.1 RecyclerView – Home Events 47
4.5.2 Discover 48
4.5.3 Following 49
4.6 Public Event View Screen 50
4.6.1 Attendance and Saving Events 50
4.6.2 Location and Links 52
4.6.3 Attendee Preview – RecyclerView 54
4.6.4 Calendar Preview – Nested RecyclerView 55

4.6.5 User's own event 56
4.7 Search Screen 57
4.7.1 Text Search 57
4.7.2 Advanced Search 61
4.8 Calendar Screen 62
4.8.1 Month View 62
4.8.2 Year View 63
4.9 Day View Screen 64
4.10 Add/Edit Private Event Screen 65
4.10.1 Event Alerts 66
4.11 Add/Edit Public Event Screen 67
4.11.1 Add Public Event 68
4.11.2 Edit Public Event 69
4.11.3 Security 70
4.12 Profile Screen 71
4.12.1 Account Information 72
4.12.2 Followers and Following 72
4.12.3 Saved Events 76
4.13 Settings Screen 78
5. Testing 80
5.1 Functional Testing 80
5.1.1 Sign-up Screen 81
5.1.2 Login Screen 82
5.1.3 Home Screen 82
5.1.4 Public Event View Screen 83
5.1.5 Search Screen 84
5.1.6 Calendar Screen 85
5.1.7 Day View Screen 86

5.1.8 Add/Edit Private Event Screen 87
5.1.9 Add/Edit Public Event Screen 88
5.1.10 Profile Screen 90
5.1.11 Settings Screen 90
5.2 User Interaction Testing 91
5.2.1 Quantitative Testing 91
5.2.2 Qualitative Testing 93
6. Conclusion 96
6.1 Learning curves 96
6.2 Challenges 97
6.3 Further Development 97
6.4 Final Thoughts 98
References 99
Appendix 102
Task 1 104
Task 2 107
Task 3 110

Table of Tables

Table 1 - Sign-in Screen Requirements 23
Table 2 - Home Screen Requirements 25
Table 3 - Public Event View Requirements 26
Table 4 - Search Requirements 28
Table 5 - Calendar Requirements 29
Table 6 - Day View Requirements 31
Table 7 - Create Private Event Requirements 32
Table 8 - Create Public Event Requirements 34
Table 9 - Profile Screen Requirements 36
Table 10 - Profile Screen Requirements (Not current user's profile) 37
Table 11 - Settings Screen Requirements 37
Table 12 - Sign-up Screen Testing 81
Table 13 - Login Screen Testing 82
Table 14 - Home Screen Testing 82
Table 15 - Public Event View Testing 83
Table 16 - Public Event View Testing (Own User's Event) 83
Table 17 - Search Screen Testing 84
Table 18 - Calendar Screen Testing 85
Table 19 - Day View Screen Testing 86
Table 20 – Add/Edit Private Event Screen Testing 87
Table 21 - Add Public Event Screen Testing 88
Table 22 - Edit Public Event Screen Testing 89
Table 23 - Profile Screen Testing 90
Table 24 - Settings Screen Testing 90

Table of Figures

Figure 1 - Skiddle Search Screen 16
Figure 2 - Skiddle Following Screen 16
Figure 3 - Skiddle Home Screen 16
Figure 4 - EventBrite Search Screen 17
Figure 5 - EventBrite Location Selection 17
Figure 6 - EventBrite Event Page 2 18
Figure 7 - EventBrite Event Page 1 18
Figure 8 - MVVM Diagram 19
Figure 9 - Firestore Subcollection Diagram 21
Figure 10 - EventDatabase Room 40
Figure 11 - PrivateEvent tableName 40
Figure 12 - EventRepository 41
Figure 13 - EventDAO 42
Figure 14 - EventViewModel 43
Figure 15 - Database Diagram 44
Figure 16 - Firebase Storage Security Rules 45
Figure 17 - Sign Up Screen 46
Figure 18 - Login Screen 46
Figure 19 - Check Email Verification Screen 46
Figure 20 - HomeEventsRecyclerView Diagram 47
Figure 21 - Home Screen (Discover) 48
Figure 22 - Home Screen (Following) 49
Figure 23 - Public Event Screen (Attendance Button Clicked) 50
Figure 24 - Public Event Screen (Attend Button Highlight) 50
Figure 25 - Profile Screen (Saved Events Highlighted) 51
Figure 26 - Public Event Screen (Attendance Set and Event in Calendar) 52
Figure 27 - Link clicked 53
Figure 28 - Location clicked 53
Figure 29 - Public Event Screen (Link Highlighted) 53
Figure 30 - Public Event Screen (Location Highlighted) 53
Figure 31 - Public Event Screen (Attendee Preview) 54
Figure 32 - CalendarPreview Instantiation 55
Figure 33 - EventViewModel Instantiation 55
Figure 34 - Public Event View (Own Users Event) 56
Figure 35 - Search Screen (Text Search) 57
Figure 36 - Search Screen 57

Figure 37 - Search Screen (Tags appearing in search) 58
Figure 38 - Create Public Event (Displaying tags) 58
Figure 39 - Search Screen (Typo Intelligence) 59
Figure 40 - API Key 60
Figure 41 - build.gradle 60
Figure 42 - Accessing API Key 60
Figure 43 - Search Screen (Advanced Search) 61
Figure 44 - Calendar Screen (Month View) 62
Figure 45 - Calendar Screen (Year View) 63
Figure 46 - Day View Screen (Event Deletion) 64
Figure 47 - Day View Screen 64
Figure 48 - Add Private Event Screen 65
Figure 49 - Event Notification 66
Figure 50 - Add/Edit Private Event Screen (Alert Highlighted) 66
Figure 51 - Add Public Event Screen 2 67
Figure 52 – Add Public Event Screen (Filled In) 67
Figure 53 - Add Public Event Screen 1 67
Figure 54 - Edit Public Event Screen 69
Figure 55 - PublicEvents Security Rules 70
Figure 56 - Profile Screen (Different User) 71
Figure 57 - Profile Screen (Own User) 71
Figure 58 - Profile Screen (Following) 72
Figure 59 - Profile Screen (Followers) 72
Figure 60 - Followers Security Rules 74
Figure 61 - Following Security Rules 75
Figure 62 - Profile Screen (Saved Events) 76
Figure 63 - SavedEvents Security Rules 77
Figure 64 - Settings Screen 78
Figure 65 - Delete Account Confirmation 79
Figure 66 - Task 1 Results 92
Figure 67 - Task 2 Results 92
Figure 68 - Task 3 Results 93
Figure 69 - Positive Feedback Word Cloud 94
Figure 70 - Application-Wide Results 95
Figure 71 - User Testing Results 104
Figure 72 - EventRepository 2 113

1.Introduction

1.1 Problem Domain

On both the Apple App Store and the Google Play Store, there are endless amounts of event apps, with event discovery being an ever present want and need for people. The focus of event ticket purchases is what governs these other applications, the approaches made by these applications are not, however, focused on the organisational aspects of managing attendance to said events.

This project aims to develop a mobile application that will better serve the organisational type of users, providing the ability to view what other people they are following are attending, what events they are hosting, and how that will fit within the user's schedule, and then publicly displaying that to their friends or family. Having a clear and well formatted User Interface/User Experience (UI/UX) is a focus of the application, which creates a user-friendly environment where the us-erbase are interconnected and have elite productivity due to the organisational prowess offered by the application for this project.

1.2 Motivations

The primary motivations for this project are:

- Creating a productivity and organisation platform where individuals can discover events, organise attendance with their friends and family, and fit these events within their schedule.
- Compiling the years of university study to test my knowledge from all aspects of my courses learning to create a product with public release potential.
- Bridging the gap between the academic theory garnered through academic coursework, with a practical application development, creating a market level product.

1.3 Aims and Objectives

The aim of this project is to create an application to help users find information on events, of many different types, the events posted by who they are following, and who is attending said events, view their calendar and their availability around that time, and then decide whether to attend said event. Users will have their own in-app calendar, where they can store private events, and public events can be added as a private to calendar also.

The objectives of this project are to:

- Perform a literature review to find the best methodology and design principles for implementing the requirements of the application.
- Analyse the event application market and obtain the positives and negatives of the competition and how that can be applicable to the application of this report.
- Construct a complete requirements analysis, laying out the entirety of the applications event discovery, organisation, and productivity features.
- Implement these features in a mobile application development language/environment by garnering new skills, especially working with APIs, online databases, and the full stack development lifecycle.
- Explore testing methods for interactive, fluid and usable UI/UX and helping in producing a market-level product by functionality testing the application.
- Find how this project could be improved and expanded on in the future with updates.

2. Literature Review

This literature review section will go over how the choices of design, methodology and implementation came into place.

2.1 Android vs IOS:

An important decision when deciding the software for this project is the type of mobile device that is most suitable for the applications needs.

One aspect that influences the decision is the demographic that both development choices would target. On one hand Android have a much greater userbase with 71.1% of smartphone users using an Android phone whereas only 28.3% use IOS(Statista, 2023). On the other hand, Apple and the App Store users tend to be much more affluent (App Data Report 2023(Business of Apps, 2023)). For this project, the affluency of the userbase is not an issue as it is planned to be distributed for free, which also tends to be what Android studio users look for.

Android development also varies from Apple in the languages and IDE that they use. Android uses Java or Kotlin to program, alongside XML for the design, whereas Apple uses Swift, a proprietary language. Swift is only available on MacOS devices, however, Android Studio is available on any system OS. In comparison, Swift is much more compact and easier to develop with than Java and Kotlin.

To conclude, due to a lack of time to develop a multi-platform application, this project will develop the application in Native Android Studio with Java and XML as the programming language as it is most suitable for

2.2 Market Research and Gap Analysis

This section analyses some competitors, their features, user interface and overall design.

2.2.1 Skiddle

Skiddle (www.skiddle.com, n.d.) is an application that offers event discovery, following of event hosts, distribution of tickets and follows several of the same objectives as this project. However, Skiddle is an event finding app that focusing purely on the sale and distribution of the tickets. Ticket sales have shown to be successful on Skiddle and 9% of ticket purchaser use Skiddle (Statista, 2022). This varies from the application of this project as it has no calendar functionality or organisational features such as viewing if people the user follows may also are attending the same event.

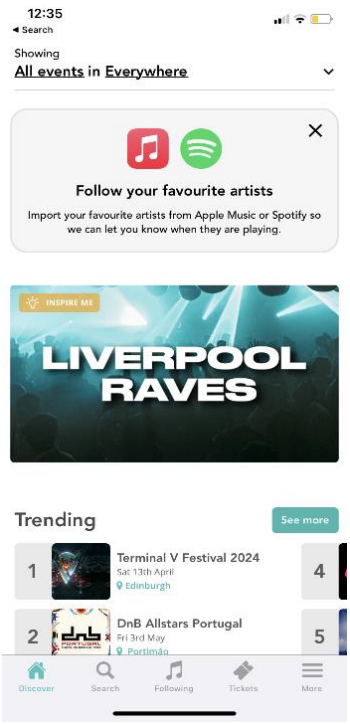


Figure 3 - Skiddle Home Screen

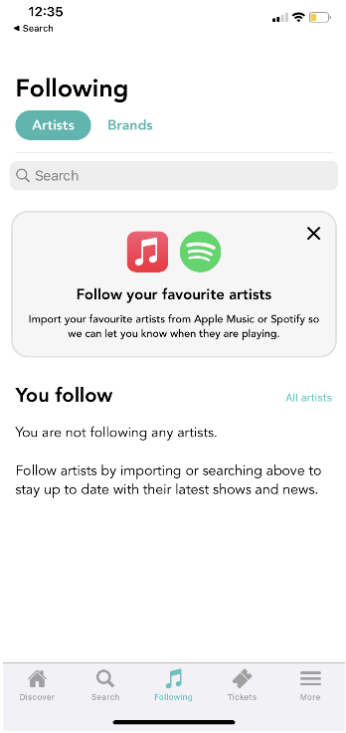


Figure 2 - Skiddle Following Screen

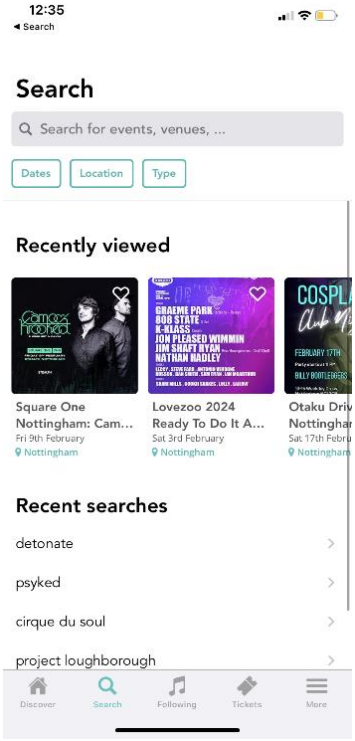


Figure 1 - Skiddle Search Screen

The positives of Skiddle are its UI/UX presentation with the big cards saying 'Liverpool Raves' in Figure 1, which captures the attention of the user, while also displaying other information with visuals such as the trending section. The top bar of Figure 3, '**All events** in **Everywhere'** is very user friendly and intuitive as two different search filters (being kind of event and location) are highlighted as it creates a sense of important to the user.

The following section, Figure 2, is a point of inspiration to this project's application, where users can follow different event artists or brands. However, it does differ from this project's application as it has the capability to import the user's favourite and following artists from Spotify and not just brands, which is an advantage over the application produced for this project. This feature is non-essential project as the application focuses more on a variety of social events, and how they can be organised by the user for productivity. Whereas Skiddle focuses more on specifically concerts, to where following specific music artists is beneficial.

In Figure 1, the search function has a search bar at the top, with the most important search filters below, which is in accordance with strong design principles (Muhammad Nazrul Islam, 2015). The use of cards with visuals is present once again with the recently viewed section and is very presentable.

2.2.2 EventBrite

EventBrite (EventBrite, 2018) is most like this project's application, as it is an event advertisement and discovery application that allows for searching of events, with filtration, and purchasing of tickets to public events. EventBrite, however, does not contain any kind of calendar functionality, with EventBrite's focus mainly being on the purchase of tickets. The goal of this project's application is viewing of other's events between different users with the events that are advertised.

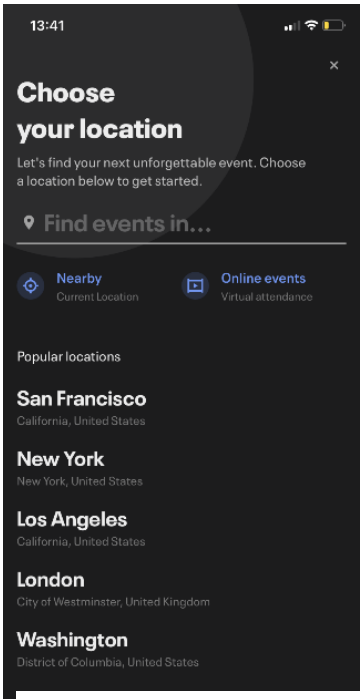


Figure 5 - EventBrite Location Selection

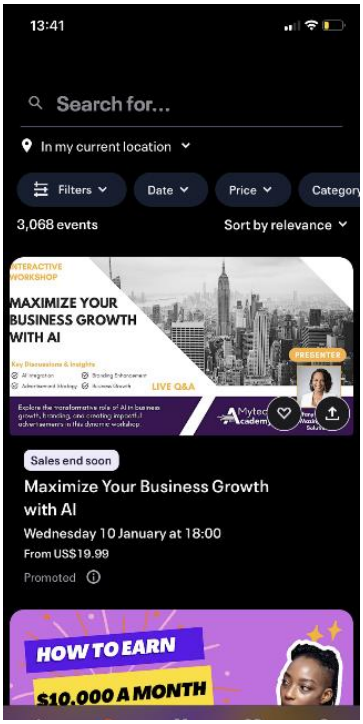


Figure 4 - EventBrite Search Screen

Figure 5 shows the screen upon first opening the application which is UX friendly as it sets the user up for the application to tailor events to the user straight away which is instantaneously engaging. Figure 4 shows the search in a similar fashion to Skiddle with the search bar at the top, followed by the important filters below.

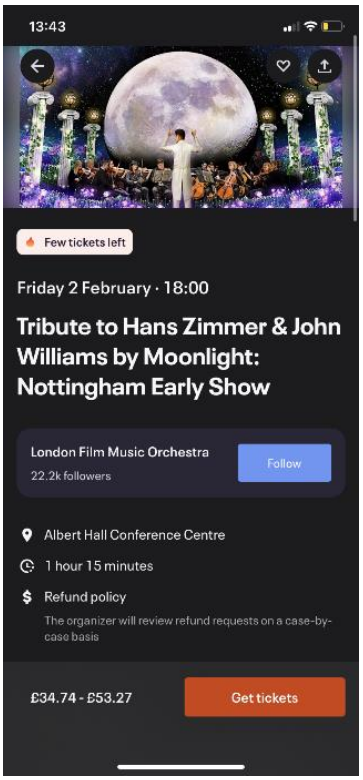


Figure 7 - EventBrite Event Page 1

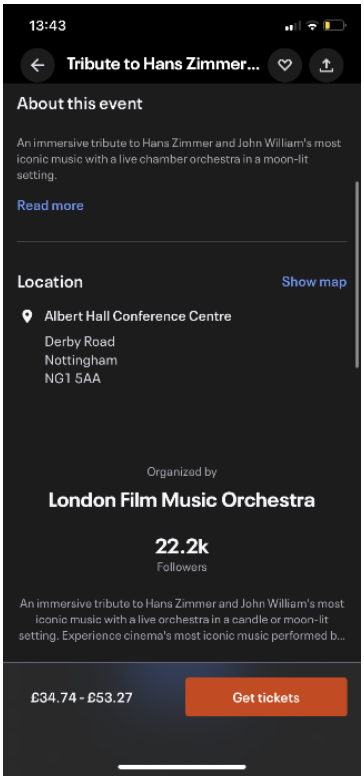


Figure 6 - EventBrite Event Page 2

Figures 6 and 7 are the event viewing page which is very impressive as it displays all information aesthetically and orderly. It also sticks to the principles of UX design as all features are accessible in 3 clicks or less. This gives me a lot of inspiration for this application's event view layout.

2.2.3 Conclusion

The gap analysis concludes that having large cards containing information of events are essential, while having search functionality allows for users to have full access to the events they require. Finally, 67.5% of attendees considering it vital for events to offer a mobile event app (Bizzabo, 2023), the event application market is flooded with endless competitors. However, the focal point of these applications is the purchase of the event tickets, where this project's application sets out with a different approach, that rather than focusing on the purchase of event, it will focus on the organisational and productivity aspect of planning their own attendance to events, which has been found to be a gap within the event discovery application market

3. Methodology and Requirements

The methodology of different technical aspects of the mobile application will be detailed in this section. This will detail the abstract concepts that have been used within the application, rather than how they are directly implemented, as that is to come within the design/implementation section.

3.1 Local Database

A local database is required for proper implementation of the application, granting the user the ability to store events directly on their device. There are a variety of Android Architectures to support a local database, the MVVM(Model-View-ViewModel), Figure 8 is the architecture imple-

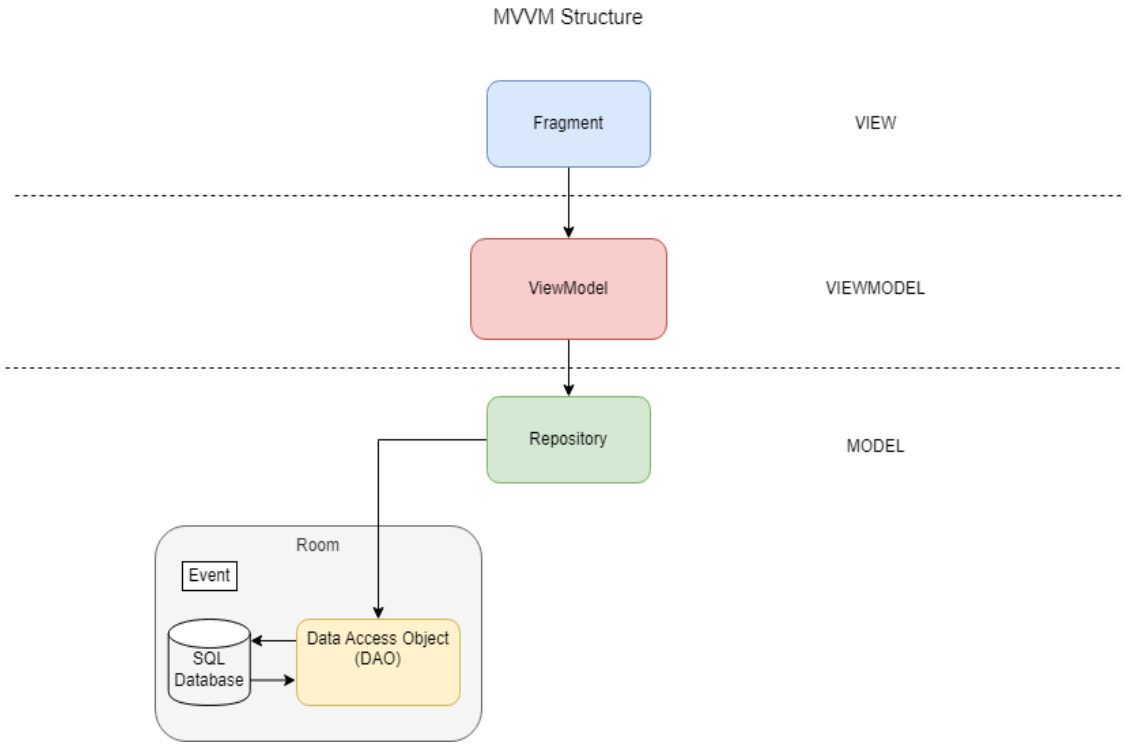


Figure 8 - MVVM Diagram

mented in the application. MVVM is composed of 3 components, where each tackle a particular function of a database and its interactivity with the UI. The MVVM architecture will be explained in the following paragraphs.

3.1.1 Model

The "*Model*" component represents the data of the system, which within Android is represented as a Room, which contains the database composed of SQLite, then directly interacted with by the Data Access Object (DAO), that can send direct queries to the database, such as delete, insert etc. The database is populated by entities (objects for Java), that creates the need for a Repository, where the application can mediate between different data sources.

3.1.2 ViewModel

The "*ViewModel"* is used for displaying data in a responsive fashion, where the repository deals with asynchronous data, this then retrieved by the *ViewModel*. The *ViewModel* works a intermediary, for where the UI retrieves data from and send updates for the data to incur. Within Android studio, the repository, uses LiveData Lists. LiveData is an observable data holder class, which is lifecycle aware, meaning it respects the lifecycle of other app components, such as activities, fragments or services, resulting in the LiveData only notifying active observers about updates. These are updated and retrieved asynchronously, creating a responsive UI, as the View will not require any refreshments for it to be updated, improving efficiency of the application.

3.1.3 View

Lastly, the *View* is the user interface, which displays the Model and sends requests to the ViewModel. The ViewModel formats the data (Model) in a way that the *View* can interpret, and as previously stated, the UI is automatically updated, as the *View* is bound to the ViewModel.

3.2 Online Database

Google Firebase is the online database chosen for this project. It offers a built-in authentication SDK with a NoSQL database structure.

3.2.1 Firebase Authentication

Firebase Authentication is built-in with a Firebase database, and offers a comprehensive backend service, complete with user-friendly SDKs and pre-made UI libraries. A key benefit is the usage of authentication in terms of security rules for the Firestore database, where the authentication of a user can be used to create rules, such as needing to be authenticated to read the *PublicEvents collection*.

3.2.2 Cloud Firestore

The type of database is the Cloud Firestore database, which has benefits over the Real-time database offered by Firebase. For example: structured data, complex querying of said data, and result-set scaling rather than collection record count scaling.

The data in Firestore is much more flexible and structured than the Real-time database, allowing for data to be kept within documents, which are within collections, while also having the capability to nest further sub-collections (with their own documents) within a document, shown in Figure 9.

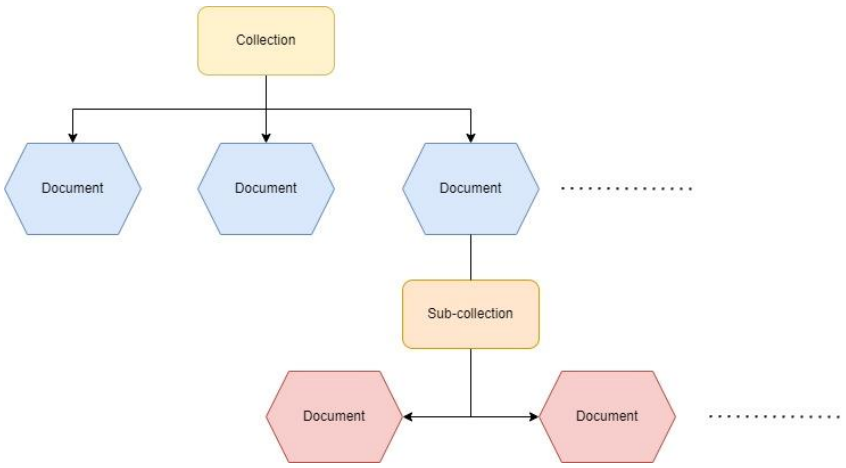


Figure 9 - Firestore Subcollection Diagram

Firestore has expressive querying which allows for multiple “where” and “order by” statements, complex queries, which are required in this applications implementation, and retrieve individual,

specific documents or retrieve all the documents in the collection. This is essential in accordance with the searching capabilities required of this application.

Firestore, not exclusively however, also allows for real-time updates, using data synchronisation to update data on any connected devices using an observer where the UI will be automatically updated with information.

Overall, following the requirements, Google Firestore is the most suitable online database for this application's needs, due to its flexibility, structure, querying capabilities, and data synchronisation.

3.2.3 Firebase Storage

Firebase also provides a way of storing files and in this project, it will be used to store images. This is done using Firebase Storage, which works within the Firebase system fluidly. The benefits of Firebase Storage include, it's scalability, robustness of uploads and downloads, and user-based security capabilities.

Firebase is a Google owned and created product and the database is built on the same infrastructure that is used with apps like Spotify and Google Photos (Firebase, n.d.), this exemplifies the scalability provided by Firebase Storage.

Uploads and downloads are also robust, meaning that as mobile users are not always online. The Firebase SDK is built to handle automatically pausing and resuming data transfer as the application loses and regains mobile connectivity.

Lastly, the Firebase SDK for Cloud Storage seamlessly works with Firebase Authentication, mentioned previously, which grants user-based security, with the declarative security model allowing access based on user authentication, or of the properties of a file. For this project it is applied to image files for the events and profiles.

3.3 Requirements

The application for this project has a detailed list of requirements, which have a description of the requirement, a scenario played out as a Stimulus/Response sequence, the functional aspect of the requirement and its priority of implementation.

These requirements have also been split into sections of the screen that implements them.

3.3.1 Sign-in Screen

Table 1 - Sign-in Screen Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Requirements	Priority
Create a user account	The user should be able to create an account	Interface should allow users to sign up with email, username, profile picture and password. User account created.	Firebase connection.	MUST
User sign in	The user can sign into an instantiated account	User inputs valid email and password, and then is taken to Home Screen	Firebase connection.	MUST
Automatic log-in	Application should log in automatically	If user closes application, without logging out. User should remain logged in.	Local database.	MUST

continued..

Requirement	Description	Stimulus/Response Sequences	Functional Requirements	Priority
Detail validation	Duplicate account details, or incorrect/insecure email and password when creating an account	User is informed of the invalidity, and what exactly is invalid. User is then prompted to try again.	None	MUST
Rejection of incorrect log-in attempt	User cannot login with incorrect/invalid account details	User enters invalid details. Application does not log-in, and alerts user of invalid details	None	MUST
Sign-up capability with third party applications	User can sign up for an account using X(formally Twitter), Google and Facebook.	User chooses to sign in with third-party app and is then prompted to input their separate details. Goes to further account creation inputs.	Third-party app interaction.	COULD

3.3.2 Home Screen

Table 2 - Home Screen Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Requirements	Priority
Discover tab	On application boot, load the discover tab, containing recently uploaded events,	User launches the application. User scrolls through given events on discover tab	Firebase connection	MUST
Following tab	A Following tab is present which displays the events of the users the current user is following. The tab must also be highlighted to display which tab the user is currently on.	User taps on the following tab, the events present change to exclusively events of users the current user follows.	Firebase connection	SHOULD
Public Events	Each event being displayed must contain their picture, title, location, start and end time, and price. On click takes the user to the appropriate public event page.	User looks at event preview information. User decides to click on the event. The event is then loaded into the public event view page.	Firebase connection	MUST

3.3.3 Public Event View Screen

Table 3 - Public Event View Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Requirements	Priority
Display title, date, description, link	The title, date, link and description of an event are all displayed on screen	User clicks on event. User can view the banner image, title, date, link and description of clicked event.	Firebase connection	MUST
Profile picture	The profile picture of the user who created the event which links to their account.	User clicks on profile picture. User is taken to the corresponding profile screen.	Firebase connection.	MUST
Link	Clicking on a link will open the default web application and is sent to website	User clicks on link. User is taken to website	Firebase connection. Internet connection	MUST
Attendance	A button will allow a user to pick between saving the event to their private calendar and/or their saved events.	User clicks on attend button User is then presented with checkboxes, which either save event publicly or add as a private event to calendar	Firebase connection. Local database	MUST
Attendee Preview	A list of people who you follow, which are also attending the event, are displayed.	User sees who follows the event, then clicks on their profile. User is then taken to the corresponding profile page.	None	SHOULD

continued..

Calendar Preview - display	At the bottom of the screen should have a calendar preview, with 3 days showing, with 1 day either side of the event day plus the event day showing.	User sees that they are free on the day of the event. User sets their attendance for the event because of this	Local database	MUST
Calendar Preview – clickable days	Each day in the preview must direct to day view of said day,	User decides they want to remove an event from their schedule or check their schedule in greater detail. User clicks on desired day to view the events on that day.	Local database	SHOULD

3.3.4 Search Screen

Table 4 - Search Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Requirements	Priority
Text search for public events	Public events can be search for by typing in their name	User types in the name of the event they wish to go to. User is presented with events similar to that name.	Firebase connection	MUST
Date searching	A date can be selected and then only events from that date will be displayed	User searches for events with a specific date. User is then shown a set of events from said date.	Firebase connection.	MUST
Minimum and max price searching	A minimum and maximum price can be set when searching for events.	User adjusts the maximum and minimum price. User searches and only events between the range are present.	Firebase connection	SHOULD
Tag searching	Any number of tags can be searched for. If an event contains any of the tags, it will appear in the search	User inputs tags. User searches and finds events matching the tags. .	Firebase connection	SHOULD
Genre searching	Event genres can be selected from to search for. Any genre can also be searched for	User selects the genre of event. User is presented with events of said genre upon search.	Firebase connection	SHOULD

continued..

3.3.5 Calendar Screen

Table 5 - Calendar Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Requirements	Priority
Have a month view of the calendar.	Able to see all the days in a month, and swap between each month through buttons at the top	User opens calendar. User sees the current month, with all the days below the corresponding day of the week.	Local database.	MUST
Long press creates pop up to add an event.	When there is a long press on a day within month view, a pop up will appear with an add event form with the date already selected	Long press made by user. Then adds event to date selected.	None	MUST
Small press opens day view	Clicking on a certain day will open into a day view of the selected day	User wants to check their agenda for a day. User then short presses a day.	None	MUST
Can view either work, social, or personal events only	A button will allow the user to switch between all different types of events or either social, work, or personal.	User wishes to see what work events they have that week and filters using the button. User is then able to decide whether to book another shift if they have or	None	COULD

continued..

		have not worked enough that week		
Have a year view of the calendar	A view of a year of the calendar, displaying all months of the years, and the year selected	User selects year. User is then able to navigate through the years and can click on a month and be returned to the month view of that month and year selection	None	SHOULD

3.3.6 Day View Screen

Table 6 - Day View Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Requirements	Priority
Have all events for the day displayed on screen	All event saved to a private user's account will be saved to the local database and displayed in chronological order.	User clicks on day and is shown all different events from said day.	Local Database	MUST
Delete events	Have a button next on the event card, where it is deleted from the database	Button is clicked. Event is removed from screen.	Local Database	MUST
Weekday selection	A bar at the top of the screen which displays the days of the week, and what day the user is on. When a day is clicked, the day view will switch to the correlative day	User clicks on a day of the week. Day view switches to said day, and displays events from that day. The weekday selection day will also highlight the clicked day	None	COULD
Add events	A button which allows a user to add an event to that day currently viewed	User clicks on the add event button. User is then directed to the private add event screen, with the date of being viewed on the day view screen, being set as for the event creation	Local Database	MUST

continued..

Editing an event	Instantiated events can be edited. Once edited, the previous event will not remain and be overwritten by newly created event.	User clicks on an already instantiated event. User is then directed to the private event edit page, with all details loaded into the	Local Database	MUST

3.3.7 Create Private Events Screen

Table 7 - Create Private Event Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Requirements	Priority
Title	Events must have a title, which has a maximum of 60 characters.	User clicks on title input. User inputs a title. If the title is over 60 characters or no title is input, then the title layout will display appropriate error message.	None	MUST
Location	Events must have a location.	User clicks on the location input. If no location is not input, then the location layout will display that there must be a location set error message.	None	MUST

continued..

Requirement	Description	Stimulus/Response Sequences	Functional Require-ments	Priority
Description	Events must have a description, which has a maximum of 30 characters.	User clicks on description input. User inputs a description. If the description is over 60 characters or no description is input, then the description layout will display appropriate error message.	None	MUST
Alerts	Events can have alerts set.	User selects out of set of possible alert, and upon event creation, alert is set for the	None	SHOULD
Date selection	A button creates a date picker.	User clicks on the Select Date button, then a date picker appears. User then selects a date and then presses okay.	None	MUST
Start and end time selection	Start and end time selector	User scrolls through minute and hour number pickers. Users do for both start and end time	None	MUST

3.3.8 Create Public Events Screen

Table 8 - Create Public Event Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Require-ments	Priority
Title	Events must have a title, which has a maximum of 60 characters.	User clicks on title input. User inputs a title. If the title is over 60 characters or no title is input, then the title layout will display appropriate error message.	None	MUST
Location	Events must have a location.	User clicks on the location input. If no location is input, then the location layout will display that there must be a location set error message.	None	MUST
Description	Events must have a description, which has a maximum of 30 characters.	User clicks on description input. User inputs a description. If the description is over 60 characters or no description is input, then the description layout will display appropriate error message.	None	MUST
Link	Events must have a link, for an organisation's website, or for a ticket purchasing website.	User clicks on the link input. If no link is input, then the link layout will display that there must be a link set error message.	None	COULD
Date selection	A button creates a date picker.	User clicks on the Select Date button, then a date picker appears. User then	None	MUST

continued..

		selects a date and then presses okay.		
Start and end time selection	Start and end time selector	User scrolls through minute and hour number pickers. Users do for both start and end time	None	MUST
Banner Image	Events have a banner image that is set by the user creating them	User clicks on add banner button, then selects the banner image. This image is then displayed.	None	MUST
Tags	Events have tags, which can be added consecutively.	User inputs a tag, presses enter and then can choose to enter another tag. If no tags are present, then an error toast message is displayed.	None	SHOULD
Genre	Events have a selection of genres which the user must pick one of.	User has a spinner which the user sets to the appropriate genre of event.	None	SHOULD
Images	Events have multiple image inputs	User chooses to add more than 1 image by pressing the more images button. Any following images are displayed on screen.	None	COULD

3.3.9 Profile Screen

Table 9 - Profile Screen Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Require-ments	Priority
User details	Have the loaded user's details displayed, including profile picture, username, and bio.	User navigates on the bottom navigation bar to the profile tab/clicks on a user's profile picture. User is then presented with either their own user details/selected user.	None	MUST
User events	All of the loaded user's events will be loaded in chronological order, with the most recent event descending from the top.	User is presented with the currently loaded user's events. User is then able to navigate to the event they desire.	None	SHOULD
Saved events	If the profile that is loaded is of the current user, then a tab will be available to show all saved events.	User taps on the saved events icon. User is now presented with the event's they have saved.	None	COULD
Settings button	A button that navigates to settings page	User taps on the settings button. User is then presented with the settings page	None	MUST

3.3.9.1 Not current user's profile

Table 10 - Profile Screen Requirements (Not current user's profile)

Requirement	Description	Stimulus/Response Sequences	Functional Require-ments	Priority
Following	A following button which indicates whether the current user follows the profile loaded.	User clicks on follow but-ton, it then changes to say following and vice versa.	None	MUST

3.3.10 Settings Screen

Table 11 - Settings Screen Requirements

Requirement	Description	Stimulus/Response Sequences	Functional Require-ments	Priority
Profile Picture reu-pload/update	A profile picture can be reuploaded and changed. Only completed once user clicks update button	User clicks on their cur-rent profile pic. User can either take a photo or upload a picture directly from their device.	Firebase con-nection	MUST
Username update	Usernames may be able to be changed, where the new username cannot be an already taken username by a sepa-rate user.	User changes username. User presses update to confirm selection	Firebase con-nection	SHOULD

continued..

Requirement	Description	Stimulus/Response Sequences	Functional Require-ments	Priority
Description update	Descriptions can be changed.	User changes description. User presses update to confirm selection.	Firebase connection	SHOULD
Confirmation of updates	A button that confirms the updates, to prevent the user from making unwanted changes	User is happy with their changes. User then confirms their choices with the update button.	Firebase connection	SHOULD
User sign out	A user can sign out of their currently logged into account, taking them back to the login screen.	User decides to log out. User presses log out button and is sent to the sign out screen.	Firebase connection	MUST
Account deletion	An account can be deleted. Where all its assets are also deleted.	User presses delete account button, where they must confirm again and re-enter their details to delete their account.	Firebase connection	MUST

4. Design and Development

This section of this project's report will detail the design thought process and application of said design to the applications implementation, firstly starting with the design and then going through each screen, and their functionalities and explanations behind choices and further detail to how they are effective.

4.1 Design Approach

This application follows a general design approach of having 2 activities, the *SignupLoginActivity* and *MainActivity*. The former deals with everything the user is capable of doing without signing in, such as account creation and signing in. The *MainActivity* is used to display the core fragments of the application, which are the screens named throughout this section, using a fragment container view. A fragment container view is the container for the fragments which are to be displayed on the screen. The bottom of *MainActivity* contains the bottom navigation bar, which switches between the five main fragments upon clicking, Home, Search, Calendar, Add/Edit Public Event, and Profile.

Having little activities and rotating through fragments throughout most of the application was an intentional design choice as it comes with several advantages. Modularity is one of the biggest advantages garnered by the fragmentation, as there are many different fragments that are reused and repurposed, such as the profile fragment for a different user's profile and the current user's profile, the sign-up fragment being reused for the settings page. With both examples, different functions are used and/or repurposed, meaning that there is less repetitive code, so greater encapsulation.

Another advantage of greater fragmentation is the encapsulation of different parts of this application's user interface, where the user has a more seamless and intuitive user experience. This is as users can navigate between different section of the application without abrupt transitions or interruptions.

4.2 Local Database Implementation

The local database as stated in the methodology section of this report, is built using the MVVM architecture. The application stores all private events, on the local database, with the logged in user only being able to view and edit events with their user id, creating a security layer for the private events of different users on the same device.

4.2.1 Model

The *Model* part of the MVVM architecture has been done using a Room database, Figure 11, in Android Studio, with *PrivateEvents* being the objects inserted as values into the table, as shown in Figure 10.

```
@Entity(tableName = "event_table")
public class PrivateEvent extends Event {
```

Figure 11 - PrivateEvent tableName

```
@Database(entities = {PrivateEvent.class}, version = 6)@TypeConverters({LocalDateTimeConverter.class, DateTypeConverter.class})
public abstract class EventDatabase extends RoomDatabase {
    3 usages
    private static EventDatabase instance;
    1 usage  1 implementation
    public abstract EventDao eventDao();

    public static synchronized EventDatabase getInstance(Context context){
        if (instance == null){
            instance = Room.databaseBuilder(context.getApplicationContext(),
                    EventDatabase.class,  name: "event_database")
                    .fallbackToDestructiveMigration()
                    .build();
        }
        return instance;
    }
}
```

Figure 10 - EventDatabase Room

The *Model* also requires a Data Access Object (DAO), to interact with the *ViewModel,* this is shown in Figure 13, which adds an extra layer of abstraction for the database, by encapsulating all access to the data source. This DAO also allows for integration with the repository layer, which is an in-between layer. This repository layer, Figure 12, acts as a mediator between the data layer and the application layer.

```java
public class EventRepository {
    8 usages
    private EventDao eventDao;
    2 usages
    private LiveData<List<PrivateEvent>> allEvents;
    1 usage
    private ExecutorService executor = Executors.newSingleThreadExecutor();


    1 usage
    public EventRepository(Application application){
        EventDatabase eventDatabase = EventDatabase.getInstance(application);
        eventDao = eventDatabase.eventDao();
    }

    1 usage
    public void insert(PrivateEvent event) { new InsertEventAsyncTask(eventDao).execute(event); }
    public void delete(PrivateEvent event){
        new DeleteEventAsyncTask(eventDao).execute(event);

    }
    public void update(PrivateEvent event){
        new UpdateEventAsyncTask(eventDao).execute(event);

    }

    1 usage
    public LiveData<List<PrivateEvent>> getAllEvents(String uid){
        allEvents = eventDao.getAllEvents(uid);
        return allEvents;
    }

    1 usage
    public void deleteEntryById(String id) {
        executor.execute(() -> {
            eventDao.deleteById(id);
        });
    }

    1 usage
    public LiveData<List<PrivateEvent>> getEventsByDate(Date date, String uid){
        return eventDao.getEventsByDate(date, uid);
    }
}
```

Figure 12 - EventRepository

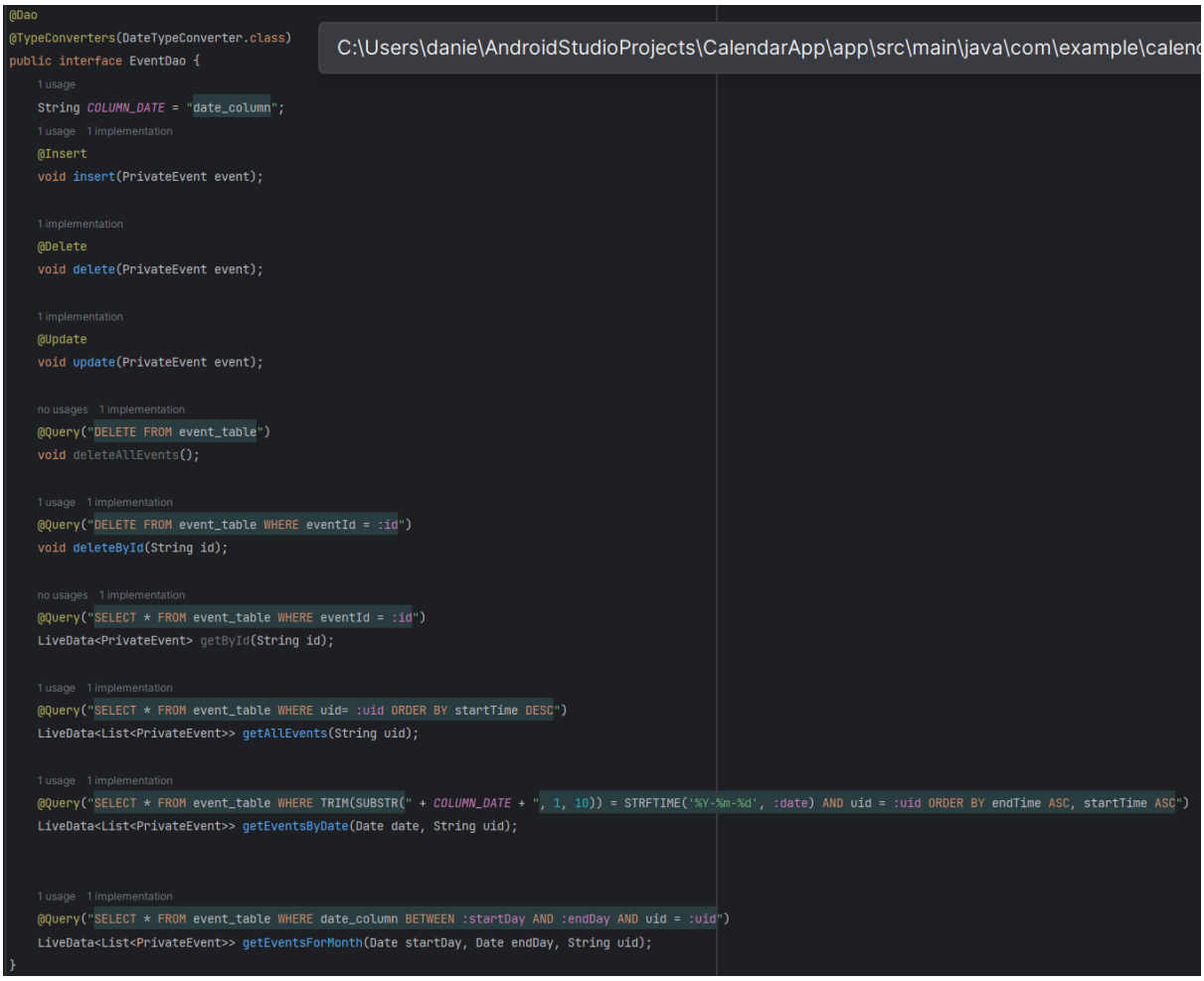


Figure 13 - EventDAO

4.2.2 ViewModel

The *ViewModel* of my application's local database is shown in Figure 14, it shows how *PrivateEvent*s are inserted, updated, and deleted, and how the retrieval of events is dependent on the uid, the current user's user id, creating the security layer between the different users of the application, meaning that you can only the view the *PrivateEvent*s of the current user.

```
public class EventViewModel extends AndroidViewModel {
    8 usages
    private EventRepository repository;
    2 usages
    private LiveData<List<PrivateEvent>> allEvents;
    2 usages
    public EventViewModel(@NonNull Application application) {
        super(application);
        repository = new EventRepository(application);
    }
    2 usages
    public void insert(PrivateEvent event) { repository.insert(event); }

    public void update(PrivateEvent event) { repository.update(event); }

    public void delete(PrivateEvent event) { repository.delete(event); }

    1 usage
    public void deleteEventById(String id) { repository.deleteEntryById(id); }


    2 usages
    public LiveData<List<PrivateEvent>> getAllEvents(String uid){
        allEvents = repository.getAllEvents(uid);
        return allEvents;
    }

    2 usages
    public LiveData<List<PrivateEvent>> getEventsByDate(Date date, String uid){
        return repository.getEventsByDate(date, uid);
    }

    1 usage
    public LiveData<List<PrivateEvent>> getEventsForMonth(Date startDay, Date endDay, String uid){
        return repository.getEventsForMonth(startDay, endDay, uid);
    }
}
```

Figure 14 - EventViewModel

4.2.3 View

Finally, the *View* component is implemented throughout the project, using observers to create dynamic *RecyclerView*s where elements are updated instantaneously as they are added, deleted, or changed. Figure 33 (calendar preview) shows one implementation of this.

4.3 Online Database

The online database, as stated in the methodology section of this report, is built on Cloud Firestore, with Firebase Storage holding any image data.

4.3.1 Cloud Firestore

Cloud Firestore is a NoSQL database, meaning not-only-SQL, which requires a different database schema prioritising the frequency of retrieval of different data. The database schema for this project's application has been optimised using this principle, having Users split up with the *Following*, *Followers* and *SavedEvents* collections being separated from the user collection, Figure 15, granting instant access for data retrieval.

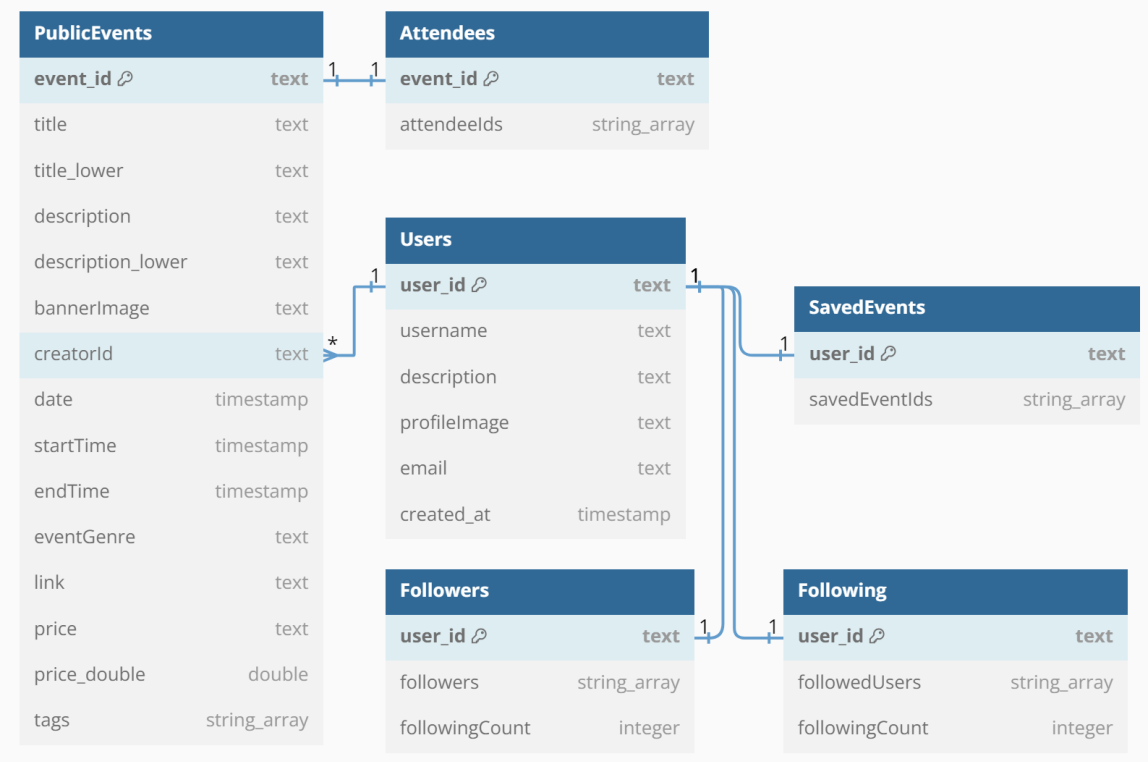


Figure 15 - Database Diagram

Figure 15 shows the relations between each collection with *Users* having the primary key, *user_id* which is associated with the foreign keys of with *SavedEvents*, *Followers* and *Following*, all having 1 to 1 relations, as each user can only have one *SavedEvents* document as it contains an array of ids they have saved. While *Followers* and *Following* can only have one document as once again they are assigned with the *user_id* as the foreign key, due to each user needing a singular document.

PublicEvents works differently, with the foreign key not being the primary key of the table, with a separate *event_id* being the primary key and the *creatorId* being the foreign key. This stores all the information of the event, and it's primary key *event_id* is the foreign and primary key for *Attendees*, which stores the user ids of users attending an event.

Security rules for this Firestore database will be disclosed in their corresponding implementation sections, Add/Edit Public Event and Profile.

4.3.2 Firebase Storage

Firebase Storage has been used to store profile images and event images. This has been used in combination with the built-in security capabilities of Firebase.

4.3.2.1 Security

The security of the storage of this application prevents users from uploading non-image files, with users not being able to upload a profile picture to *profileImages* without their user id being that of the document id, line 10 in Figure 16, and for *eventImages* there has been metadata, *creatorId*, stored for every upload of an event image, where this must match the user id which uploads the event creator, line 23.

```
3   service firebase.storage {
4     match /b/{bucket}/o {
5
6
7       match /profileImages/{userId}/{allPaths=**} {
8
9         // Allow write if the user is authenticated, userId matches the user's ID, and the file is an image.
10        allow write: if request.auth != null && userId == request.auth.uid + '.jpg'
11        && request.resource.contentType.matches('image/.*');
12
13        // Allow read if needed, can be restricted further based on your app's requirements
14        allow read: if request.auth != null;
15      }
16
17
18        match /eventImages/{eventId}/{allPaths=**} {
19
20          // Allow write and update if the user is authenticated, the file is an image
21          // and the metadata of the image is correct users id
22          allow write: if request.auth != null && request.resource.contentType.matches('image/.*')
23          && request.resource.metadata.creatorId == request.auth.uid;
24
25          // Allow read if needed, can be restricted further
26          allow read: if request.auth != null;
27        }
28    }
```

Figure 16 - Firebase Storage Security Rules

4.4 Sign-up/Login Screen

Firebase Firestore is the online database used within the application, which provides a built-in Authentication SDK, where the user can sign in with an email address and password. The Firebase Authentication SDK is vital in how the user's permissions within the application are dealt with, for example only an authenticated user with the same user ID as their "PublicEvent" document, can alter and delete said document. This stops nefarious parties from altering documents with correct permissions.

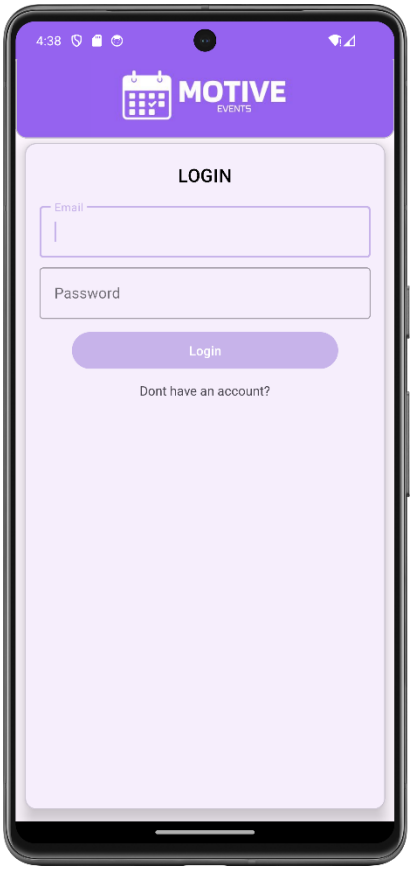


Figure 18 - Login Screen

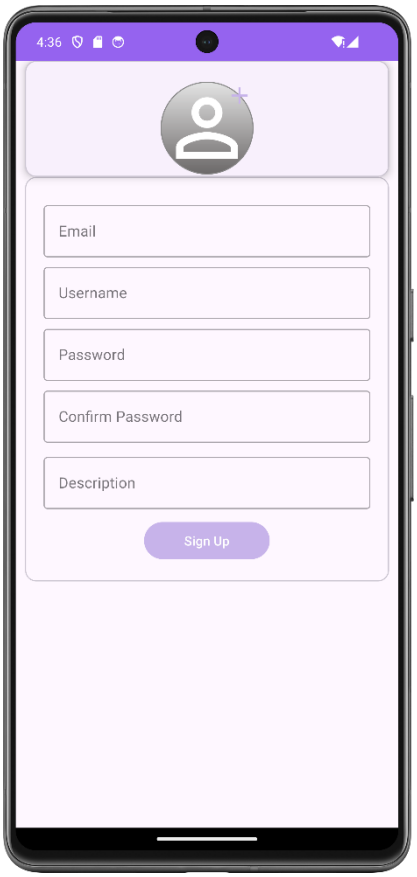


Figure 17 - Sign Up Screen

Figure 19 - Check Email Verification Screen

The application has a sign-up screen where the user must input their email address, username, password, confirm that password, and a description of their account. The confirm password must match the first password. A profile picture is also required for account creation, where the user clicks on the profile picture icon at the top and can either take a photo or upload directly from the device.

Upon inputting valid information, the user will then be sent an email verification link, where once completed, will be directed to the main activity, with the Home Screen loaded initially.

4.5 Home Screen

The Home Screen implements a key component for this project's application, a RecyclerView, which displays the events in a dynamic ordered list. A RecyclerView was implemented alongside the Firebase to create a live list of PublicEvents.

4.5.1 RecyclerView – Home Events

A RecyclerView is an AndroidX ViewGroup component which creates dynamic lists of custom components. There is a custom layout, *item_home_event*, which is then handled by the *RecyclerView* adapter *HomeAdapter*, which takes events, which have been retrieved by *the PublicEventRepository*, and binds each event to a position within the adapter, the event data (e.g. title, location, etc) is bound to the *ViewHolder*, for this instance *HomeEventHolder*, by the *RecyclerView*. Another aspect of a *RecyclerView* is the *LayoutManager* defined for each *RecyclerView* which determines the organisation of the display of the *ViewHolders*, the layout chosen for the Home Screen is the *LinearLayoutManager*, which arranges items in a one-dimensional list, set to the vertical orientation. This *HomeEventsRecyclerView*, Figure 20, implemented on the application's home screen will reoccur throughout the application as it is the main *RecyclerView* format used for displaying PublicEvents.

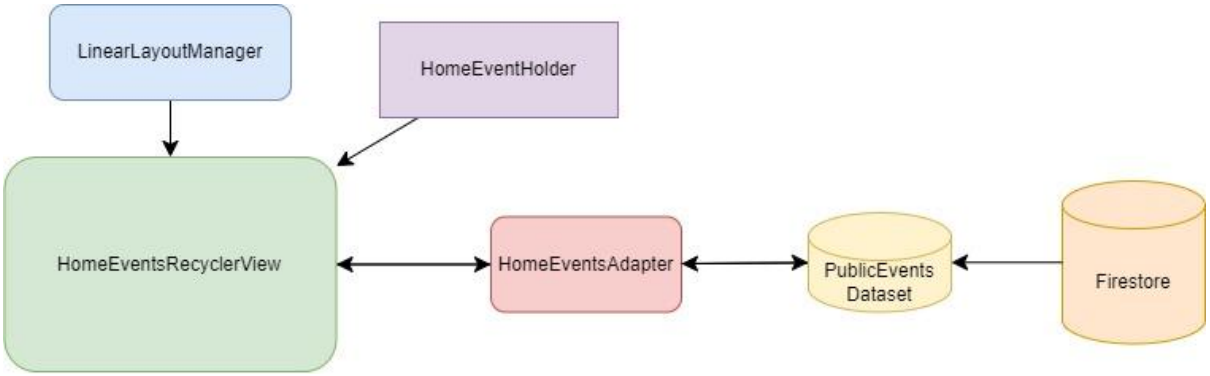


Figure 20 - HomeEventsRecyclerView Diagram

4.5.2 Discover

The Home Screen offers 2 key functionalities, Discovery, and Following. The prior is the display of all PublicEvents, displayed in a descending order of upload date and time, so displaying the most recently uploaded events at the top in descending order. This grants users a discovery option of all the newest events, so they can be the first to view and set their attendance or organise their schedule around said event, granting the user organisational and productivity optimality, which is the main objective of this project.

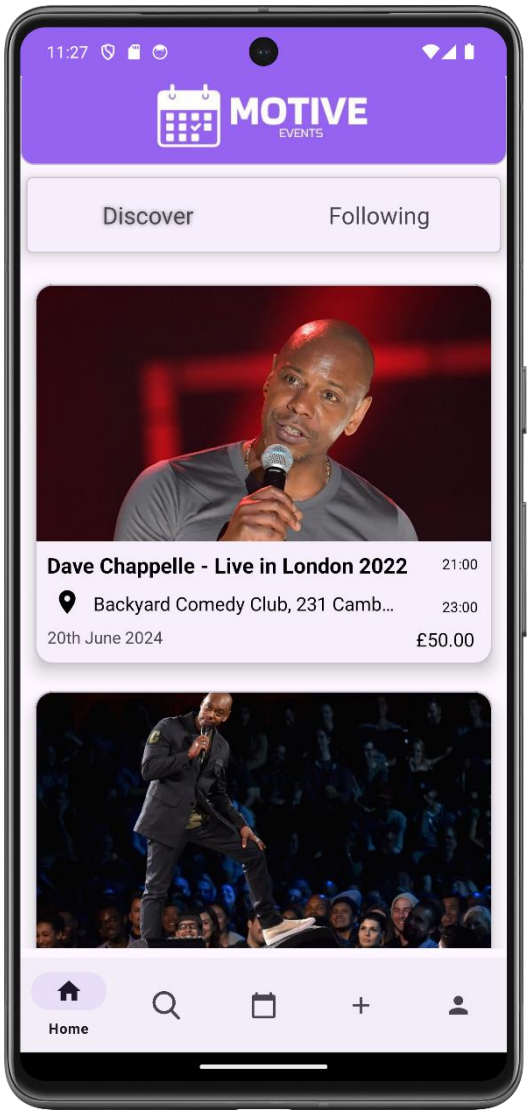


Figure 21 - Home Screen (Discover)

4.5.3 Following

The Following tab offers another key feature to the user, displaying the events that have been uploaded by the users that they follow, which is essential for the following system to work as intended. Users can follow other user's accounts and have exclusively the users they follow's events in this feed.

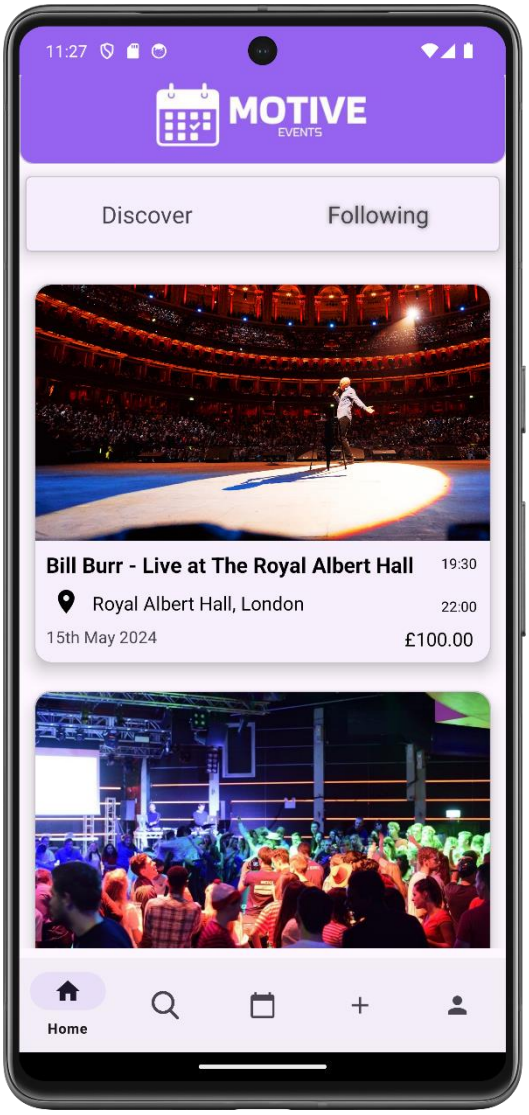


Figure 22 - Home Screen (Following)

4.6 Public Event View Screen

The objective of this screen is for the user to be presented with the event's information, and the calendar around the date of the event, as to show whether the user is free around the date and time of the event.

4.6.1 Attendance and Saving Events

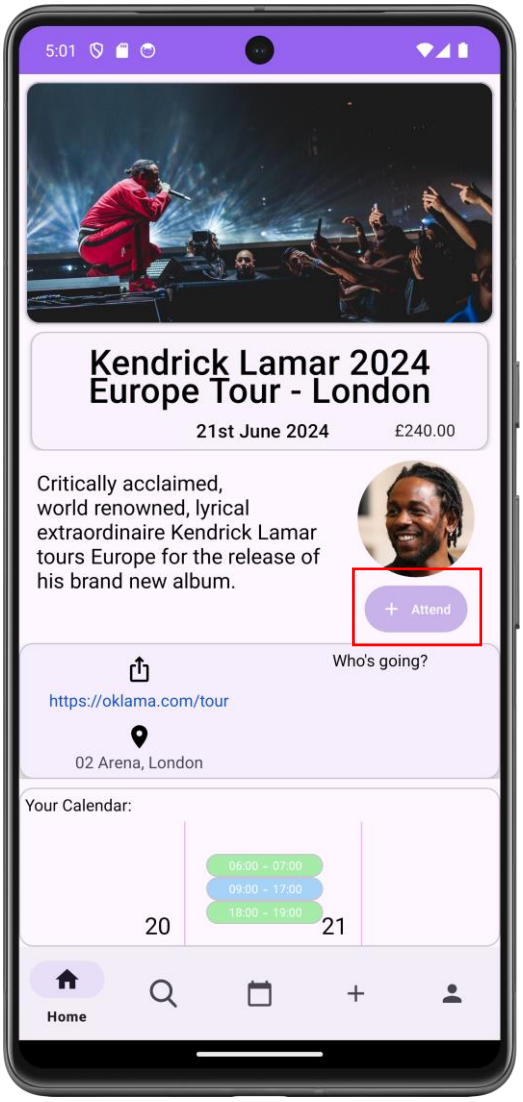


Figure 24 - Public Event Screen (Attend Button Highlight)

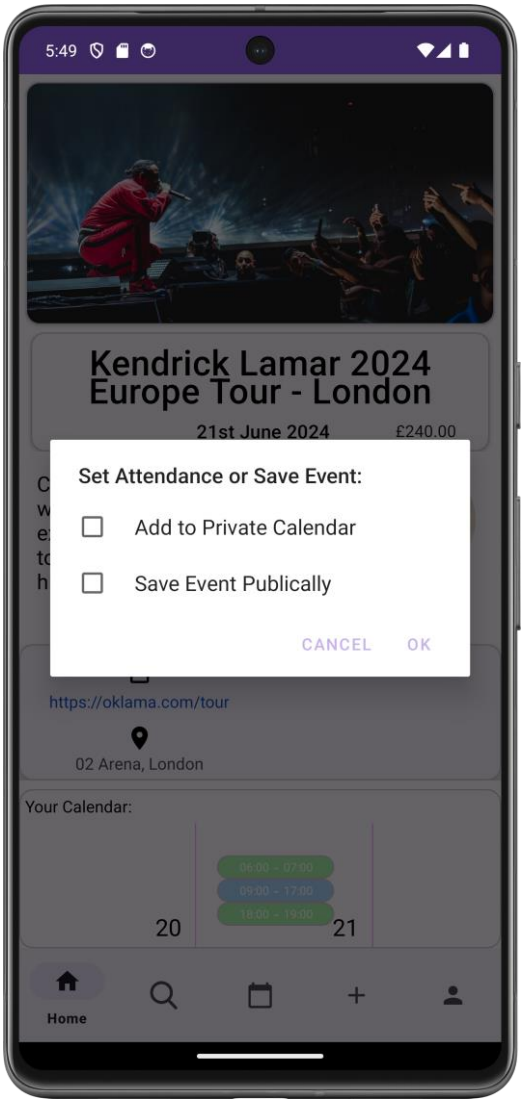


Figure 23 - Public Event Screen (Attendance Button Clicked)

The attend button creates a checkbox dialog box, shown in Figure 23, where the user can select between setting their attendance by adding the event to their private calendar, or saving the event to their saved events, which can be found on their home page (Figure 26).

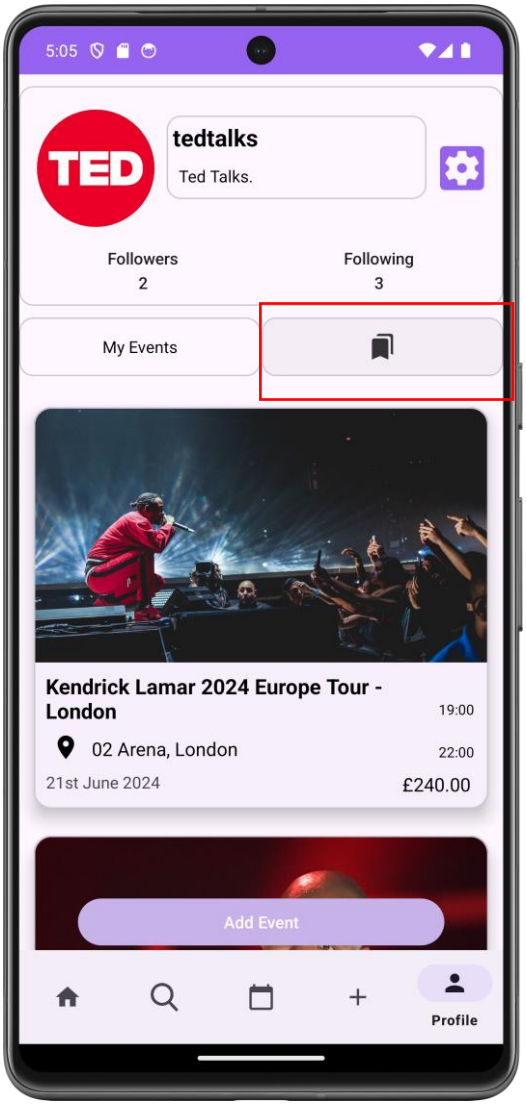


Figure 25 - Profile Screen (Saved Events Highlighted)

4.6.1.1 Attendance

Attendance is stored on the online Firestore database, and when the user attends something, it will be added to the separate *Attendees* collection, with the eventId being the document id. When attendance is set, the user will see the update of the calendar preview, Figure 25, showing that the event has been added to their private calendar. Only once added to the private calendar, will the attend button switch to attending button, with the same functionality (Figure 25).

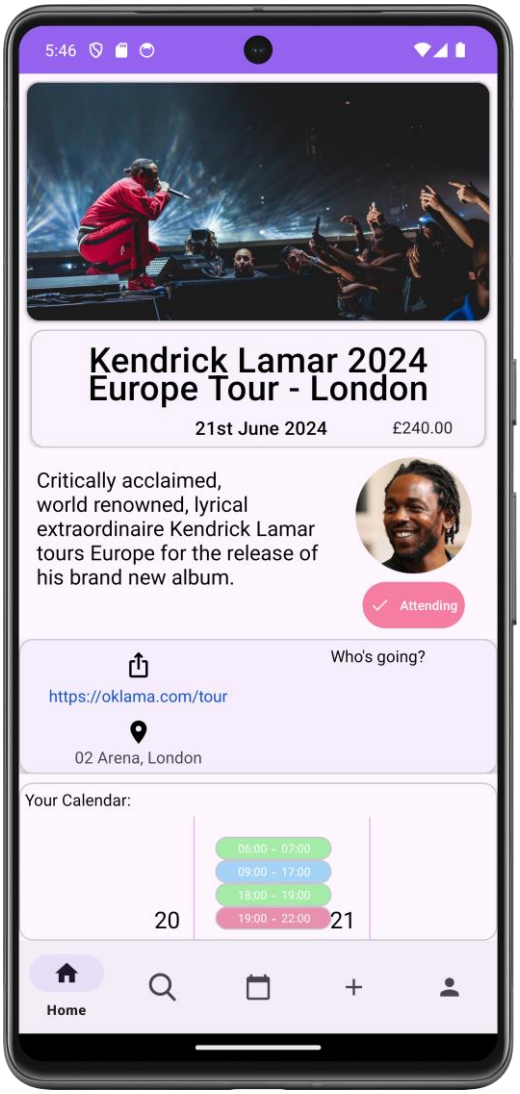


Figure 26 - Public Event Screen (Attendance Set and Event in Calendar)

4.6.2 Location and Links

Locations are displayed (Figure 30) and if clicked by the user, they have the location opened in Google Maps (Figure 28). The same for the links that are displayed (Figure 29 and 27).

Figure 30 - Public Event Screen (Location Highlighted)

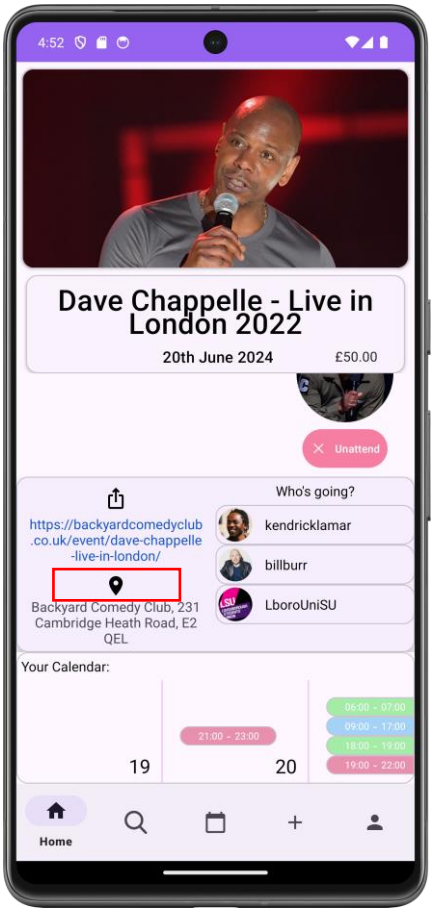
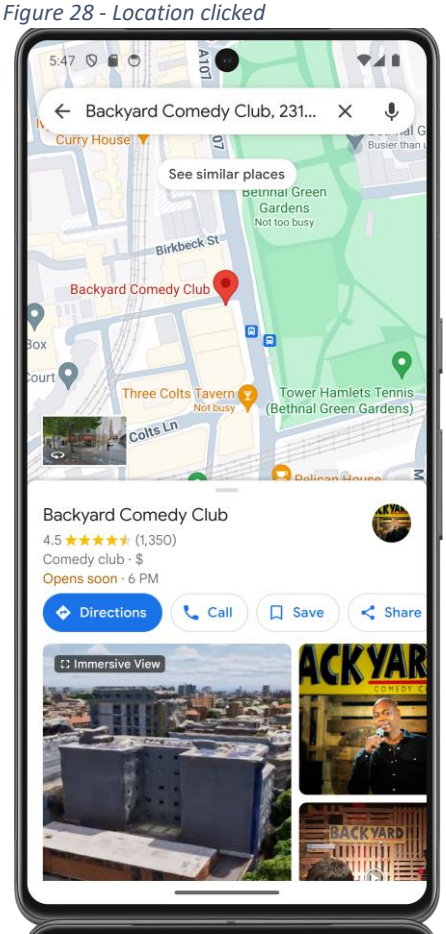


Figure 28 - Location clicked



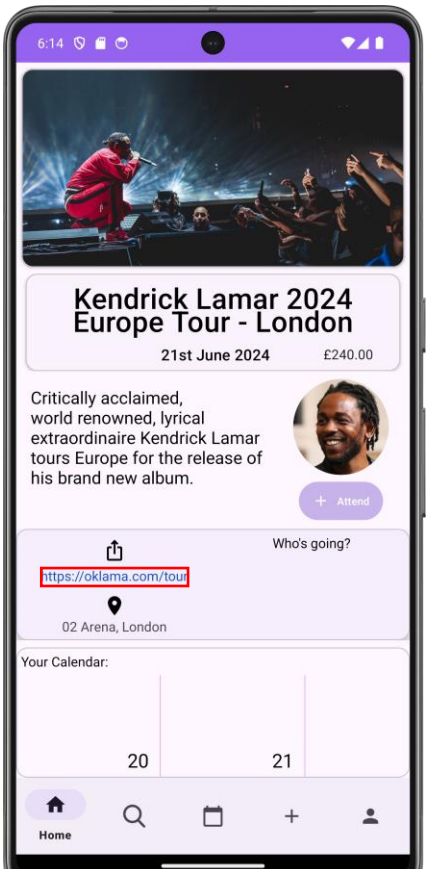


Figure 29 - Public Event Screen (Link Highlighted)

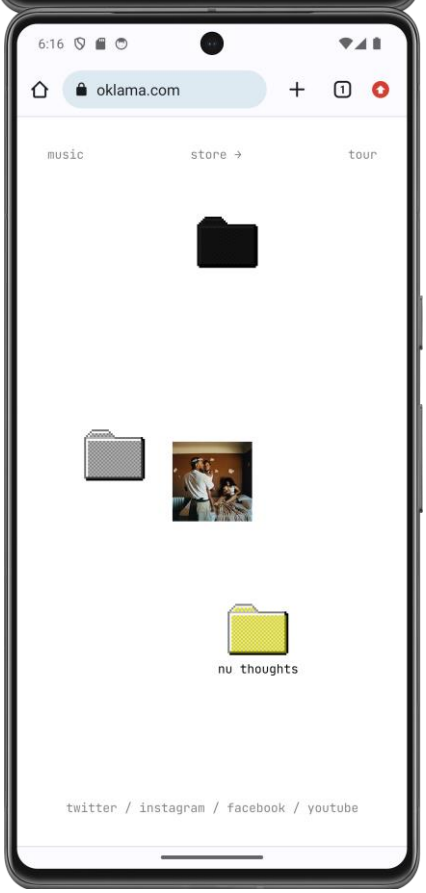


Figure 27 - Link clicked

4.6.3 Attendee Preview – RecyclerView

A RecyclerView is used to display all the attendees that the user is following, including their username and profile picture. The previews are clickable as well, taking the user to the profile page associated to the preview. This is essential for the organisation brought by the application as it will give users awareness of what of the people they follow are attending, so they are able to know whether they should attend themselves.

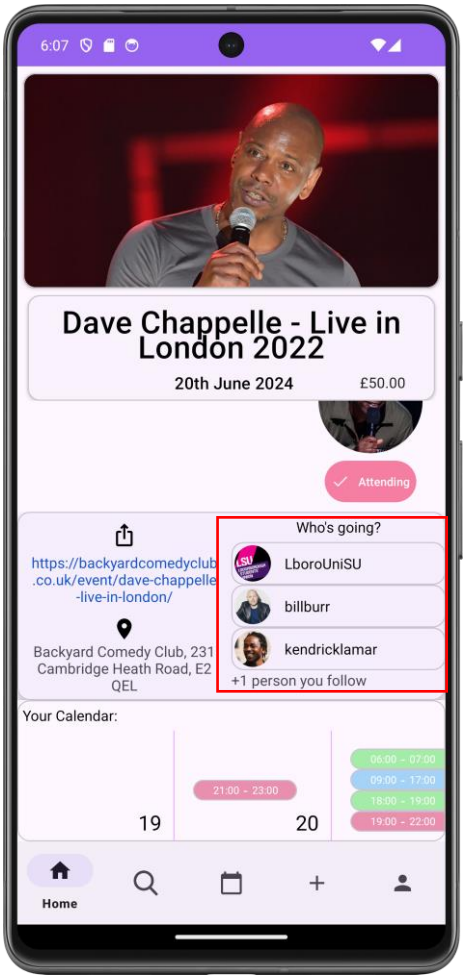


Figure 31 - Public Event Screen (Attendee Preview)

The preview shows at max 3 previews and if there are more than 3 attendees the user follows, then it will display how many (Figure 31).

4.6.4 Calendar Preview – Nested RecyclerView

The calendar preview in this instance is constructed with a nested *RecyclerView*, a *RecyclerView*, inside another *RecyclerView*. The external *RecyclerView* sets the calendar preview layout, and the internal *RecyclerView* displays the list of events for the day in which it is nested inside.

4.6.4.1 External RecyclerView's adapter, CalendarPreviewAdapter

Instantiating a *RecyclerView* within another can only be done by passing particular parameters into the external adapter, *CalendarPreviewAdapter*; them being an *Application* object, the application, using *getApplication*(), the *LifecycleOwner* of the current fragments view using *getViewLifecycleOwner(), Figure 32*. These unusual parameters for a RecyclerView are essential for it's dynamism and nested capabilities.

```
Application application = requireActivity().getApplication();
calendarPreviewAdapter = new CalendarPreviewAdapter(daysAroundDate, itemListener, monthYear, getContext(), date, application, getViewLifecycleOwner());
```

Figure 32 - CalendarPreview Instantiation

Firstly, the Application must be passed as it is the component that allows for retrieval for the application's ViewModel, (Figure 33) is from the *CalendarPreviewAdapter*, which grants the *CalendarPreviewAdapter* access to the applications *Model*, the database's data, where the adapter can retrieve the private events for needed for the day of the year which is to be displayed in the

```
eventViewModel = new EventViewModel(application);
eventViewModel.getEventsByDate(currentDate, user.getUid()).observe(lifecycleOwner, privateEvents -> {
    for(int i=0;i<privateEvents.size();i++){
        Log.d( tag: "Event found in month", privateEvents.get(i).toString(), tr: null);
    }
    privateEventPreviewAdapter = new PrivateEventPreviewAdapter(privateEvents);
    holder.eventPreviewRecyclerView.setAdapter(privateEventPreviewAdapter);
    Animation animation = AnimationUtils.loadAnimation(application.getApplicationContext(), R.anim.float_in);
    holder.eventPreviewRecyclerView.startAnimation(animation);
});
```

Figure 33 - EventViewModel Instantiation

calendar preview. The Application is also used for garnering the context, of the application, which makes the animations possible.

Secondly, the *LifecycleOwner*, the current view's *LifecycleOwner* being the Fragment, *PublicEventViewFragment*, which is a mirror of said Fragment's lifecycle. This is once again vital for the *ViewModel* instantiation, with *.observe* needing the *LifeycleOwner* to detect when the Fragment is ended, so as to stop the observation.

4.6.4.2 Internal RecyclerView's adapter, PrivateEventPreviewAdapter

The *PrivateEventPreviewAdapter* is the nested *RecyclerView*'s adapter which is bound to the ViewHolder of the *CalendarPreviewAdapter*'s *RecyclerView*, *CalendarPreviewHolder*. This is a relatively simpler adapter, which inflates it's ViewHolder, *EventPreviewHolder*, and determines the type of event and sets the colour of the *EventPreviewHolder* to that colour and determines the start and end type of each event input, and then sets the correct times to give the user enough preview information, while not overcrowding the view and making it unreadable. The use of the ViewModel within the internal adapter, grants UI efficiency through the page not needing to be reloaded to display the newly instantiated private event in the calendar preview upon attendance being set or unset.

4.6.5 User's own event

When the event is the user's own event, then there are some changes, the absence of the profile picture, the change of the attend button to the edit button, and the disappearance of the calendar preview, as it is not needed for your own established event (Figure 34).

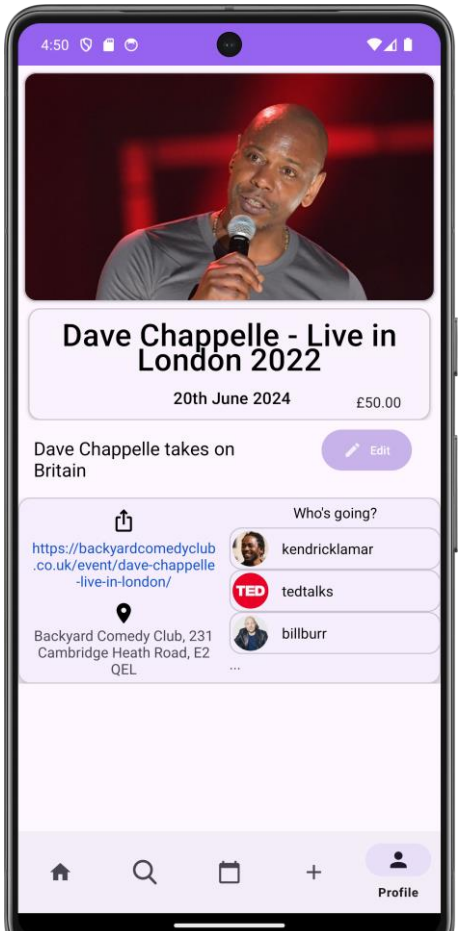


Figure 34 - Public Event View (Own Users Event)

4.7 Search Screen

The search screen contains two main functionalities, text search querying, where the user can input text to find events, and the more advanced search with tags, price ranges and event genre.

Figure 36 - Search Screen

Figure 35 - Search Screen (Text Search)

4.7.1 Text Search

Text Search is a feature where there needs to be a level of comprehension above basic Firestore querying, even if the client-side composite querying is useful. That is why the application implements the Algolia Text Search API, which is a very powerful tool for text search of an entire collection's documents.

4.7.1.1 Algolia Search Benefits

Algolia is fantastic for UX, as it has 3 extra capabilities not accessible with Firebase's inbuilt querying, instant search, balance relevance and it handles typos intelligently (Algolia, 2023).

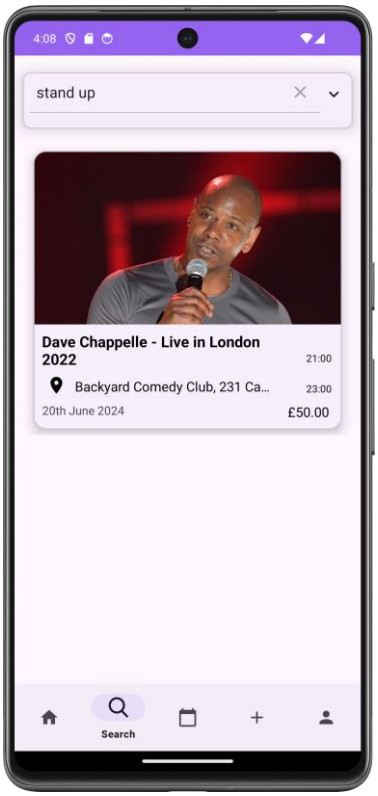


Figure 37 - Search Screen (Tags appearing in search)

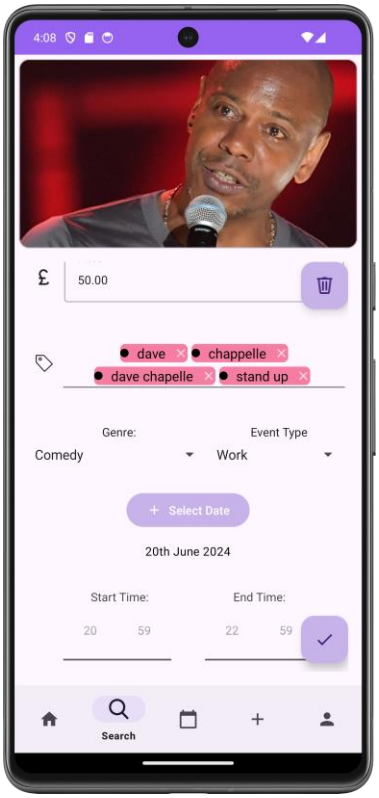


Figure 38 - Create Public Event (Displaying tags)

Instant search refers to how the index-wide results are presented and updated every time a key is entered, in real time (Algolia, 2023). Searching across the entirety of each *PublicEvents* document's fields, brings sophisticated searching, not possible with Firestore querying. As shown, in Figure 37 and Figure 38 tags can be found within the search results.

Balancing relevance and popularity on search are impossible with Firestore, as there is no way to filter or order search results by search results, so for this application, if the user were to type in Bill, and there were 2 events with Bill in their event document, then the event will the most requests will take precedence and be returned at the top.

Lastly, typo intelligence is vital for good search UX, as if the user is to type in the desired word or phrase with Firestore querying, then the user then must locate their typo and then fix it. This is solved with Algolia, as it provides out of the box typo-tolerance, that works on both words and prefixes, as shown in Figure 39.

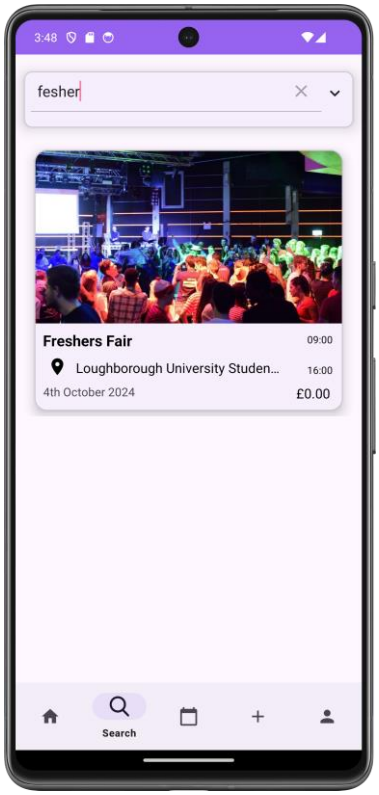


Figure 39 - Search Screen (Typo Intelligence)

4.7.1.2 API Key Security

Working with an external API requires an API key for the application to access it's services. This API key however must be hidden and not released to the public. This is as to stop service abuse, where an exposed API can use the search engine outside of the controlled environment of the application, meaning that there is a potential of large volumes of requests, leading to either greater charges by the API provider, or denial of service attacks, which can overwhelm the API server.

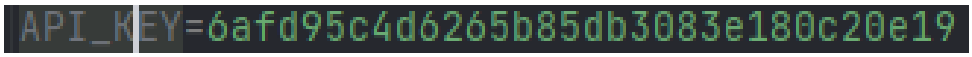


Figure 40 - API Key

```
val localPropertiesFile = rootProject.file("local.properties")
if (localPropertiesFile.exists()) {
    localPropertiesFile.inputStream().use { it: FileInputStream
        localProperties.load(it)
    }
}

val apiKey = localProperties.getProperty("API_KEY") ?: "No API Key"
buildConfigField( type: "String",  name: "API_KEY",  value: "\"$apiKey\"")
```

Figure 41 - build.gradle

The *local.properties* file is used to store the API key(Figure 40), and have then accessed that through the defaultConfig section of the *build.gradle* file(Figure 41).

```
private final String API_KEY = BuildConfig.API_KEY;
```

Figure 42 - Accessing API Key

The key is then retrieved by the *SearchFragment* class from the *BuildConfig* (Figure 42). All of this is necessary to create a security layer, preventing malicious attacks.

4.7.2 Advanced Search

Firebase is useful because of the advanced querying capabilities, where, once indexed in the Google Cloud Platform, the client can query with a combined set of indices. Within the application, there is search enabled for minimum and maximum price, tag searching for up to 10 different tags, and genre searching, or for any searching (Figure 43)

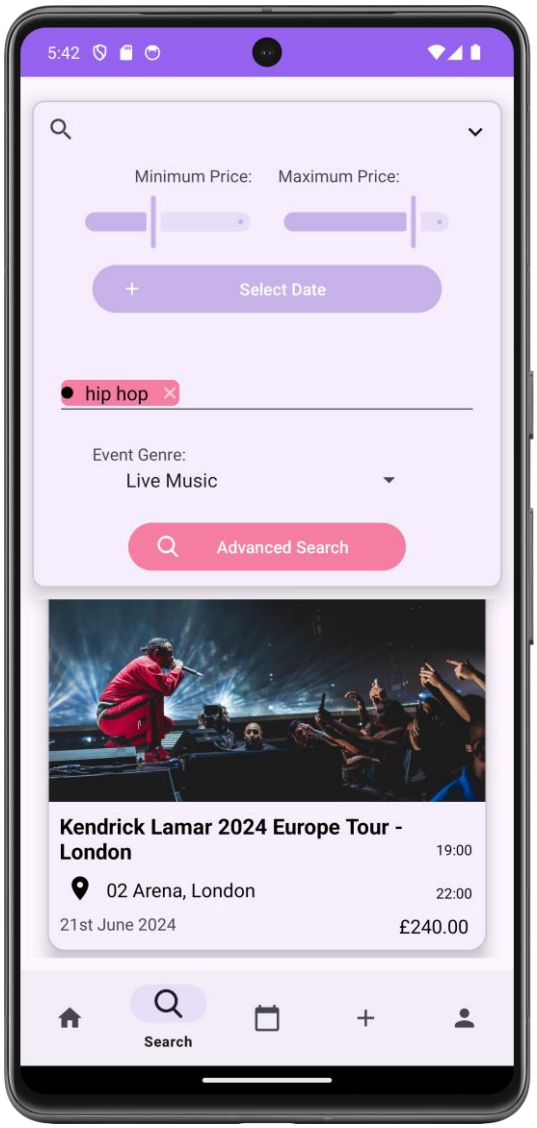


Figure 43 - Search Screen (Advanced Search)

4.8 Calendar Screen

Having a built-in calendar which interacts dynamically with the private and public events of the application was key to this project's success. Creating a calendar which displays the events of a day, allows the users to plan for any upcoming events.

4.8.1 Month View

When first opening the calendar tab, the month view of the calendar is shown, which displays the current month and year at the top, with a grid layout *RecyclerView* displaying the days of the month, where each column representing a day of the week and each row represents a week of the month (Figure 44). This uses a very similar function to the calendar preview on the public event view; however, it does not display the start and end times of the events as there is not enough screen space.

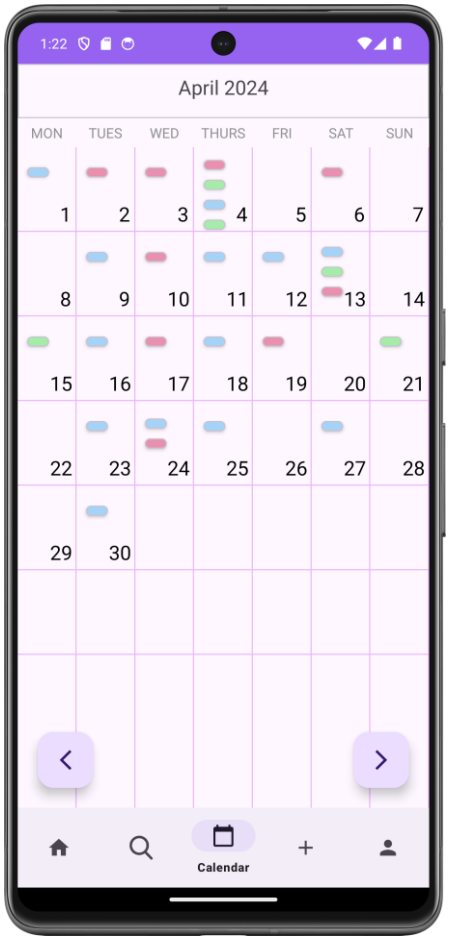


Figure 44 - Calendar Screen (Month View)

If the month and year text at the top of the screen is clicked, then the user is directed to the year view. If a day of the month is clicked, then the user is taken to the Day View Screen.

4.8.2 Year View

A simple RecyclerView displays the months of the year, and with the year currently selected at the top. The back and forward buttons now working as buttons that cycle through the years (Figure 45). The month view of a selected month and year will be shown upon clicking the desired month.

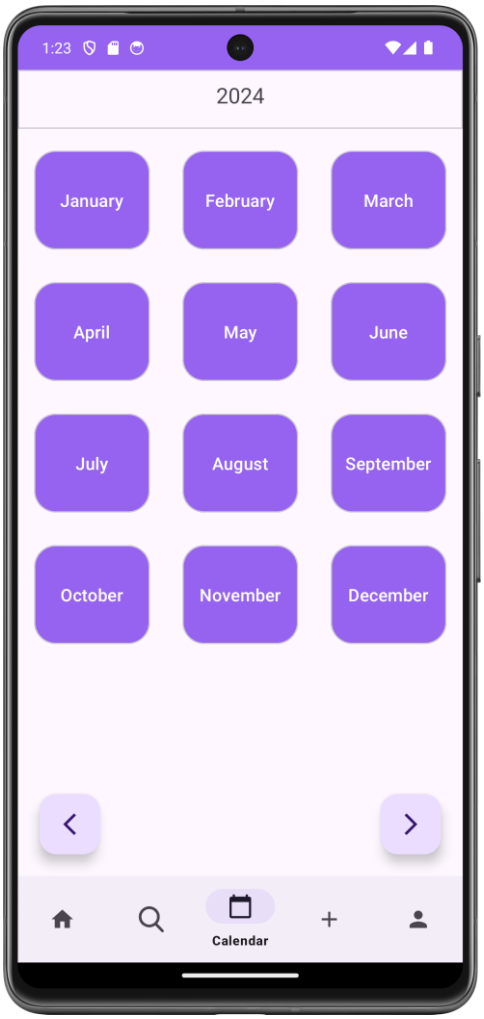


Figure 45 - Calendar Screen (Year View)

4.9 Day View Screen

The Day View Screen (Figure 47) is a simple RecyclerView, with all the events saved to the local database, *Model*, observed using the ViewModel, once again, and displayed on the *View*. By

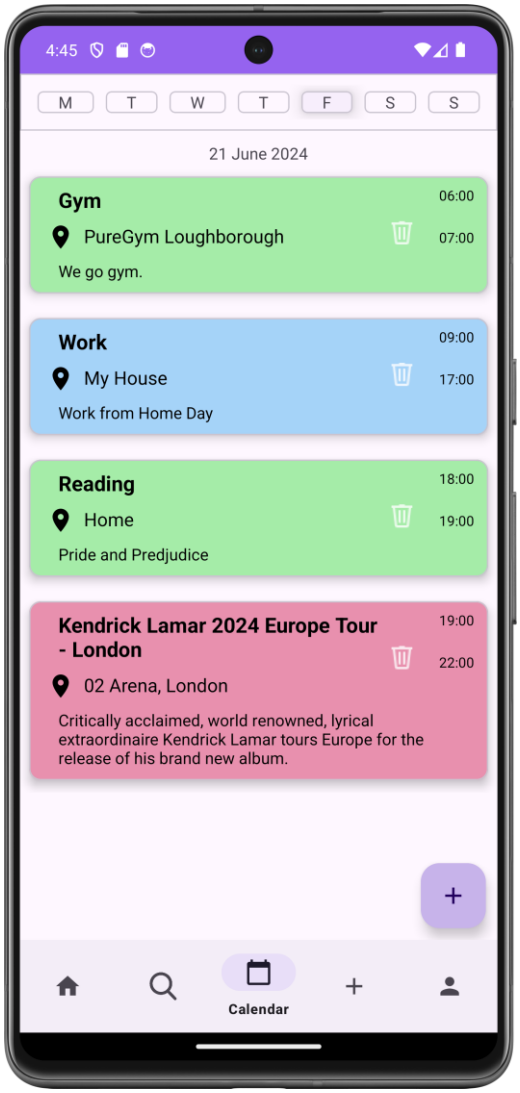


Figure 47 - Day View Screen

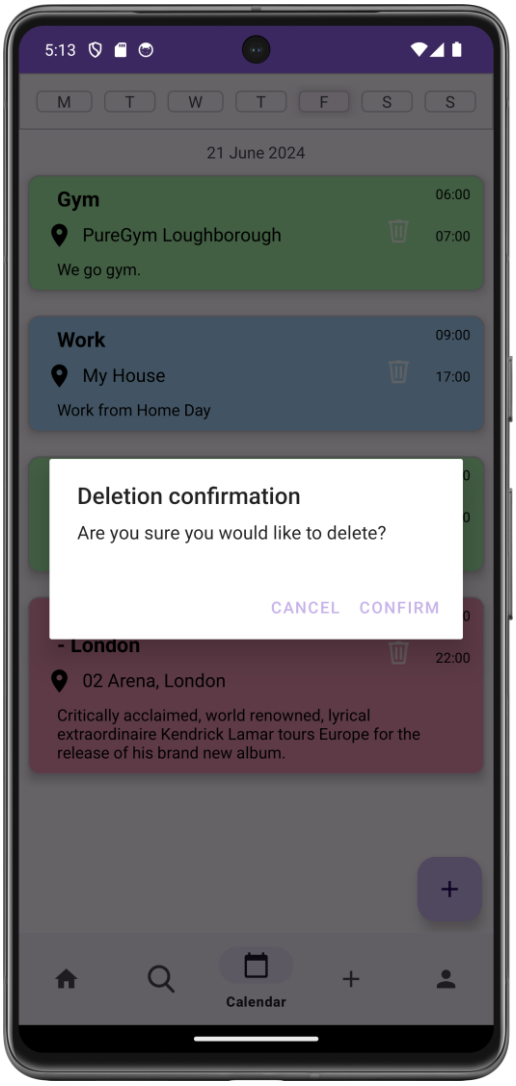


Figure 46 - Day View Screen (Event Deletion)

clicking on the event, the user is taken to the Private Event Add/Edit Page, where the event clicked on is loaded. There is a delete button for each event, enabling easy deletion of any event. Where the user is presented with a pop-up asking whether they wish to delete the event, as to stop accidental deletion of events (Figure 46).

There is also a list of the days of the week at the top of the screen, which highlights the current day of the week selected, and when clicked will switch to that day, of the current week the day being viewed is in.

4.10 Add/Edit Private Event Screen

Adding a private event is essential for the quick addition of non-public events to the calendar as it means that users can achieve their desired organisation, by not only representing the public events they are attending, but also the events within their private life.

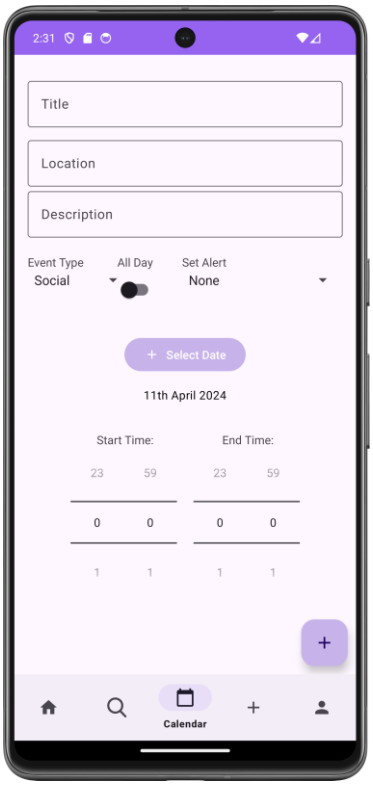


Figure 48 - Add Private Event Screen

A private event contains a title, location, description, event type, an option of an alert, a date and a start and end time. The event type (social, work, personal) determines the colour of event, shown in Figure 48.

Users can also edit their events by clicking on them within the day view screen, which changes the plus symbol to a tick (Figure 50), as to signify to the user that they are overwriting a preexisting event.

4.10.1 Event Alerts

Private events can have alerts set for 5 minutes, 15 minutes, 30 minutes, 1 hours or 2 hours before the event occurs (Figure 49). These reminders will display a notification, with the title of the event and the time of the event (Figure 50).

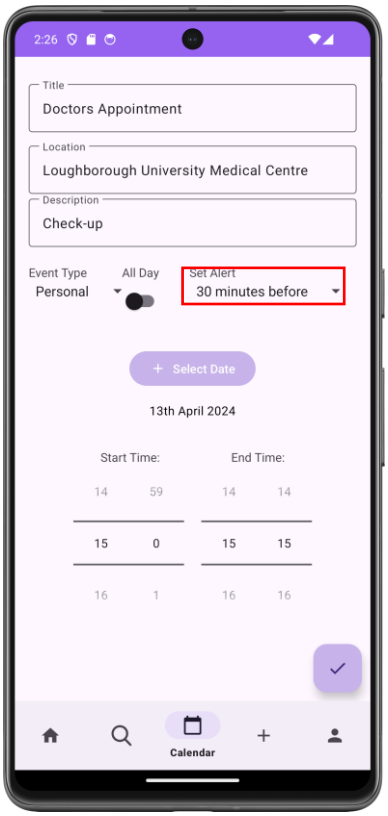


Figure 50 - Add/Edit Private Event Screen (Alert Highlighted)

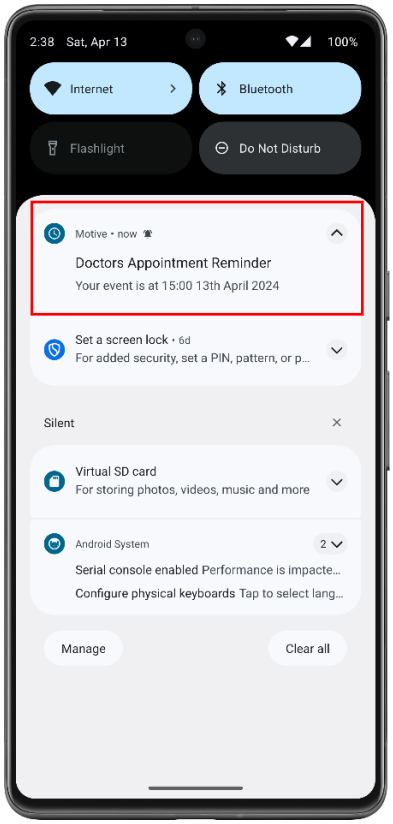


Figure 49 - Event Notification

This is implemented by the *ReminderBroadcast* class, which receives an intent, and tailors the reminder to fit the event needed. A switch case is used to determine how long before the start time should the reminder be set.

4.11 Add/Edit Public Event Screen

A public event is composed of a banner image, title, location, description, link, price, a minimum of 1 tag, genre (live music, conference, etc), type (social, work, personal), date, and start and end times. Tags are implemented using the *TagsEditText* library (Abbas, 2024). The Add/Edit Public Event screen, Figure 51, 52, 53, facilitates the upload of these events to the Firebase Cloud Fire-store online database.

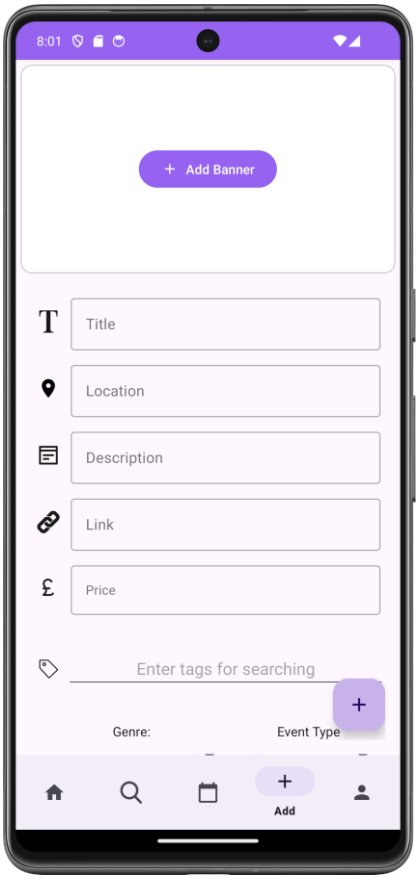


Figure 53 - Add Public Event Screen 1

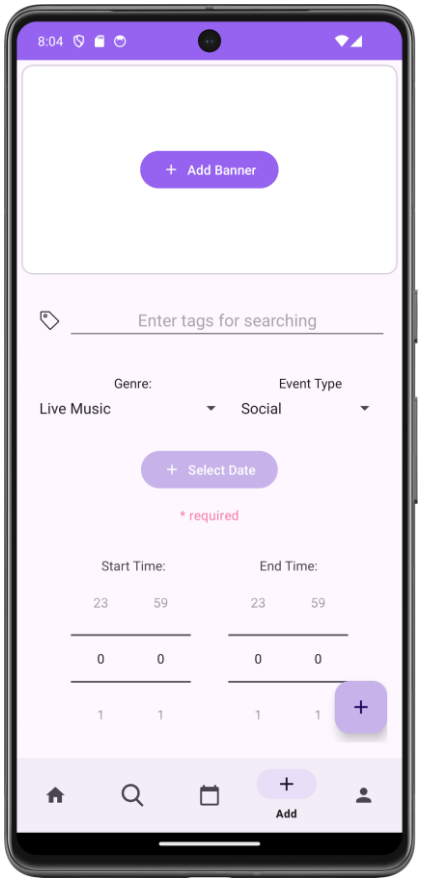


Figure 51 - Add Public Event Screen 2

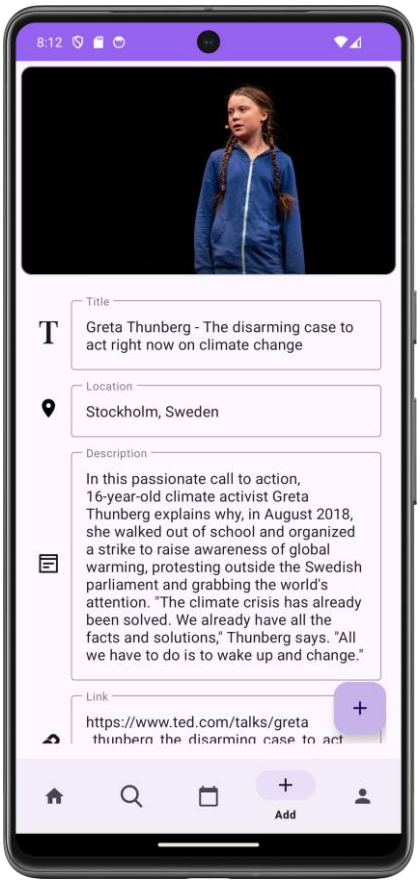


Figure 52 – Add Public Event Screen (Filled In)

4.11.1 Add Public Event

4.11.1.1 Firebase Storage

The same application of Firebase Storage's features that are applied during the sign-up process, where the user must be authenticated to upload an eventImage. The application sets the metadata for that image, to have the creatorId be the user's id, as to prevent other users from changing an event not created by themselves.

4.11.1.2 UX

This project's key component, Public Event distribution, is of utmost importance, so there is UI/UX theory applied in this case, which states that UI should attempt to append meaningful small images or thumbnails with linguistic signs; and append an appropriate icon with the linguistic sign, where necessary to amplifying the meaning of interface sign (Muhammad Nazrul Islam, 2015). This has influenced the decision to give the text inputs icons, along with constant text prompts of what is to be input in each field. This is done by using vector image icons next to the input fields and using the Material Components library's *TextInputLayout* and *TextInputEditText*. The *TextInputEditTexts* contain the hint and when entered that hint is displayed in the *TextInputLayout*, as shown in (Figure 52).

4.11.1.3 Image Selection

I have used an external library to implement the image selection functionality. It is a combination of the *ImagePicker (Patel, 2022))*, and *Glid (bumptech, 2019)* librarys. ImagePicker creates a popup asking the user for a photo from their camera, or an image from their device. This is then placed within the ImageView by the Glide library.

4.11.1.4 Date Picker

Material Components has a built in DatePicker, which creates a pop-up calendar, for the user to select the desired date. This then is retrieved as a Date object and passed as a parameter for the construction of the PublicEvent object.

4.11.2 Edit Public Event

If the user clicks on the edit button on the PublicEventView screen, which only available when the user is the creator of the event, the event's data is loaded into the views on the page (Figure 54). The user is then able to edit any data of the event and reupload, replacing the previous event.

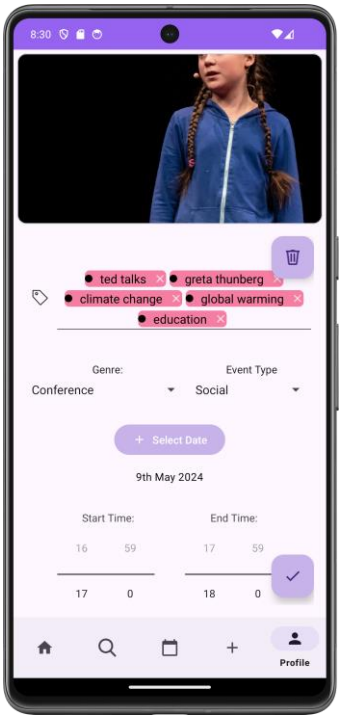


Figure 54 - Edit Public Event Screen

The add button is changed from a plus symbol to a tick symbol to indicate the update of the event rather than the addition of a new event. There is also a new delete button present, which deletes the event from the Firestore database.

If the user does not press the confirm button (the tick button), then the updates made to the event will not be saved as per usability standards.

4.11.3 Security

Firestore Rules are used to create barriers of entry to certain aspects of data retrieval and manipulation within the online database. The *PublicEvents* collection contains the documents with all the information pertaining to each event, within each document, with each document's id being that of the event id. These rules are show in Figure 55.

```
19      match /PublicEvents/{eventId} {
20        // Allow any authenticated user to create an event, and the creatorId must be the user's id
21        allow create: if request.auth != null && request.resource.data.creatorId == request.auth.uid;
22
23        // Allow anyone to read events
24        allow read: if request.auth != null;
25
26        // Allow only the user who created the event to update or delete it
27        allow update: if request.auth.uid == resource.data.creatorId
28              // only the current user's id can ever be set as the creatorid
29                && request.resource.data.creatorId == resource.data.creatorId;
30
31        allow delete: if request.auth != null && request.auth.uid == resource.data.creatorId;
32      }
```

Figure 55 - PublicEvents Security Rules

These rules describe that only authorised users can create an event, and where the user id associated with the event must be that of the user creating it. This ensures that there will not be events listed by other users, claiming that it was created by current user. The update rule is the same, however with the difference that the *resource.data.creatorId* must be the users id, which is different from the create rule's *request.resource.data.creatorId*. This is different as when there is the request prefix, it is a descriptor of the data that is being requested to be written, which is what must match the create rule as there is no resource not from the request. Whereas for the update, there is a *creatorId* already set, and that cannot be changed, and it is read from the resource already written.

Additionally, any authorised user can read the PublicEvents, and users can only delete their own events.

4.12 Profile Screen

The profile screen, Figure 57, is used for displaying the user's profile upon clicking on the profile tab on the bottom navigation bar, or for when a user clicks on another user's profile. If it is the current user's profile that is loaded, then a settings button is displayed which takes the user to the settings screen.

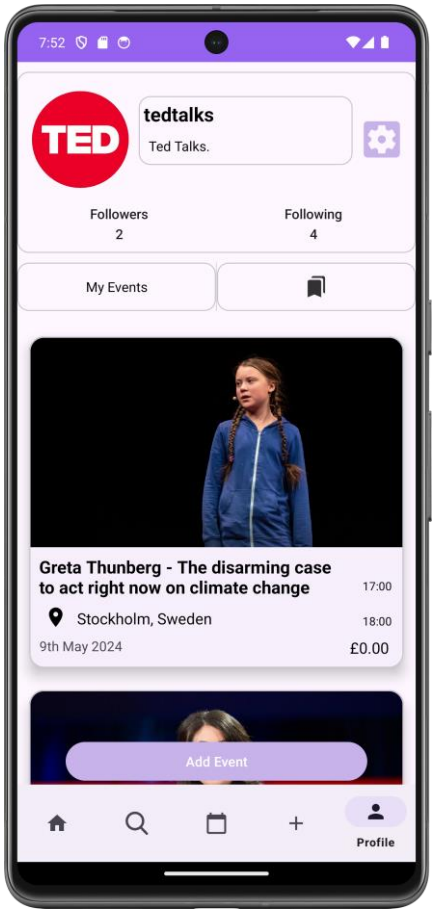


Figure 57 - Profile Screen (Own User)

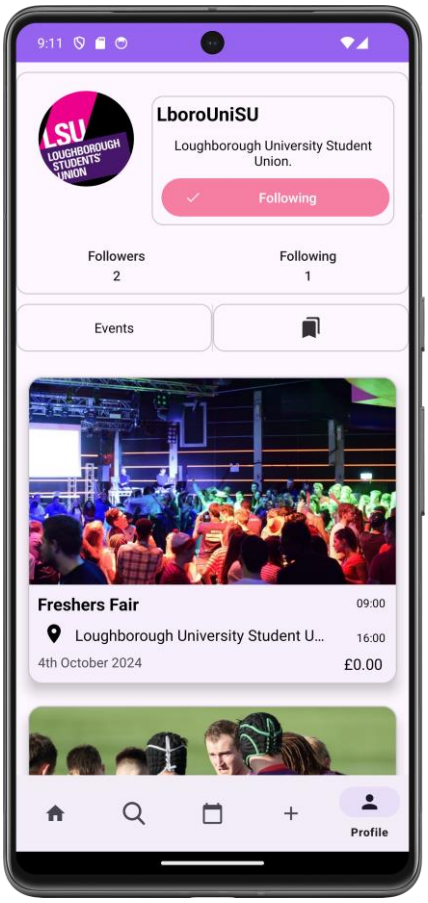


Figure 56 - Profile Screen (Different User)

If the profile is not the current users, there is no settings button however there is a follow button which indicates whether the user is followed, and when clicked switches between following and follow (Figure 56).

4.12.1 Account Information

The user's profile that is loaded displays important information about the user, such as their profile picture, username, bio. If it is not the user's account, then a following button is present which indicates whether the user follows said user, and if clicked toggles between following and not following the account (Figure 57).

4.12.2 Followers and Following

Below the initial information is the following and follower counts, which once clicked on provide the user with a list of the users the account they are viewing either follow or are following the account (Figure 58 and 59).

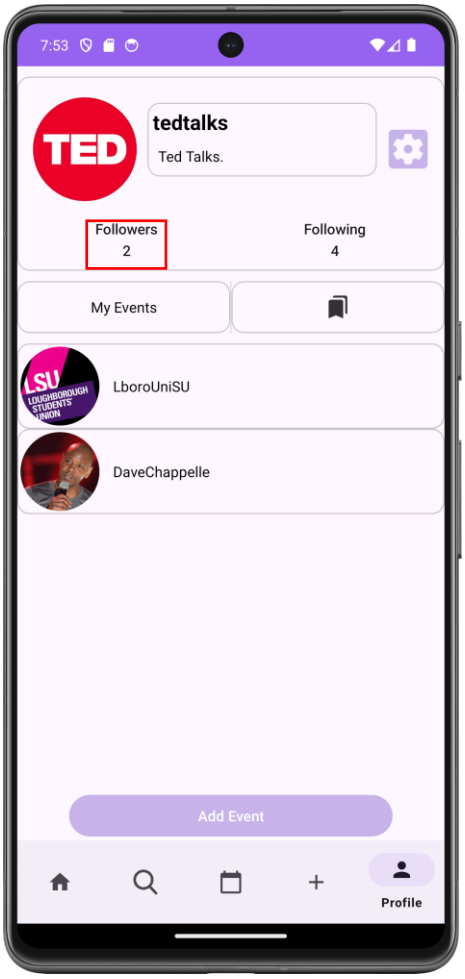


Figure 59 - Profile Screen (Followers)

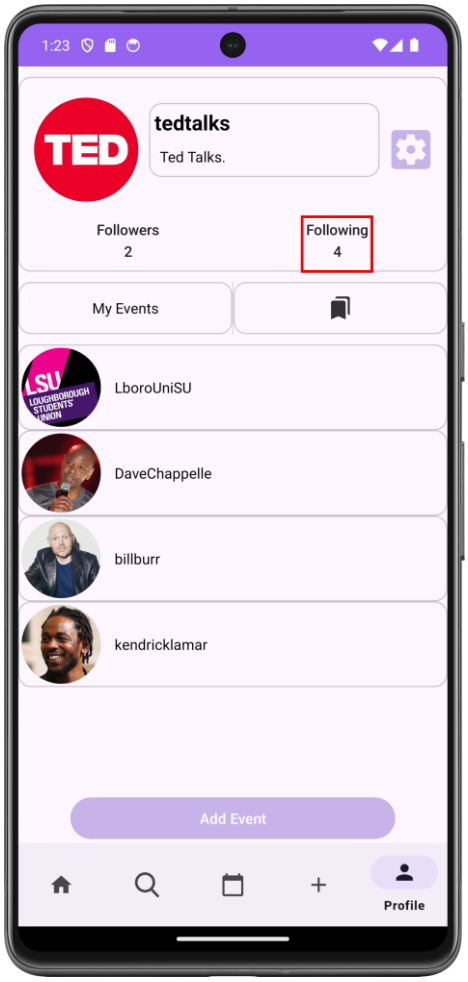


Figure 58 - Profile Screen (Following)

This uses two collections in Firestore, *Following* and *Followers*. Both collections store arrays of user ids, which function as pointers to the *User* documents that they correspond to. Having separate collections in *Following* and *Followers* allow for ease of search for the user's followers or following, as stated in the decision making behind the Firebase NoSQL, as if there was no *Followers* collection, then to find the followers of a user would require the search of all the documents in the *Following* collection for the desired user's user id.

This is sub-optimal for scaling and performance of the application, when committing a singular read and two writes, when writing twice is more efficient than one write and lots of reads whenever any user needs to find their followers or following (Silberschatz, 2019). This also allows for a follower count to be recorded directly in the same collection's documents, so only one document needs to be accessed, making the singular read much more efficient.

4.12.2.1 Security

Firestore security rules are vital when securing the data for the database, as they dictate who can create, read, update, or delete different documents. (Figure 60) shows the rules for the *Followers* collection.

```
match /Followers/{followingId} {
  allow create: if request.auth != null && request.resource.data.followers.hasAny([request.auth.uid]);

  // Allow any authenticated user to read who's following who
  allow read: if request.auth != null;

  allow update: if request.auth != null
    && request.resource.data.followers is list
    && (
      (
        // Adding their own user ID (only their ID can be added)
        request.resource.data.followers.size() == resource.data.followers.size() + 1
        && request.auth.uid in request.resource.data.followers
        && !(request.auth.uid in resource.data.followers)
      ) || (
        // Removing their own user ID (only their ID can be removed)
        request.resource.data.followers.size() == resource.data.followers.size() - 1
        && !(request.auth.uid in request.resource.data.followers)
        && request.auth.uid in resource.data.followers
      )
    );
  // Allow delete if the user trying to delete the document is the same as the creator
  // This assumes your documents include a 'creatorId' field for who created the follower document
  allow delete: if request.auth != null && isOwner(request.auth.uid, request.resource.data.followerId);
 }

}
```

Figure 60 - Followers Security Rules

These rules declare that any user can read the followers of another user.

Any user can update the followers list, which is the list of followers within the document, but only if they are adding their own user id to the list, and the list increases by one, and where their user id is not currently in the list. Or if they wish to remove themselves from another user's followers, then the size of the list of followers must only decrease by one, where the new list does not contain their user id and the previous list did contain their user id.

This has been done to prevent malicious users from changing users' followers without their permission.

The corresponding *Following* collection (Figure 61) has documents with the document id being the id of the user which is following other users, and the array of *followedUsers* in the document is the ids of the users that they follow. Only users who match the *request.auth.uid* will be able to instantiate their own following document, this makes it essentially easier to work around than with the Followers collection. All users can read the following of other users.

```
54      match /Following/{followerId}{
55
56        // Allow any authenticated user to follow someone
57        allow create: if request.auth != null && request.auth.uid == followerId;
58
59        // Allow any authenticated user to read whos following who
60        allow read: if request.auth != null;
61
62        // Allow only the user who created the event to update or delete it
63        allow update: if request.auth != null && (request.auth.uid == followerId) ||
64        (request.auth.uid == request.resource.data.followedUsers);
65
66        allow delete: if request.auth != null &&
67          (request.auth.uid == followerId ||
68          request.auth.uid == resource.data.followedUsers);
69
70      }
```

Figure 61 - Following Security Rules

With updating a Following document, on the user who follows someone, may update the document, which follows along with the same rule as the create rule, however if user's id is a *folowedUser*, then they may edit the document also update the document.

4.12.3 Saved Events

The saved events tab, Figure 62, displays the events saved by the user. This is done by clicking the attend button on the Public Event View Screen, and then clicking the “Save Event Publicly”. Then when any profile is viewed, the user can view the saved events, to see what events the user is going to. This also doubles as useful for the user’s own profile, with the user being able to view all the public events they are attending.

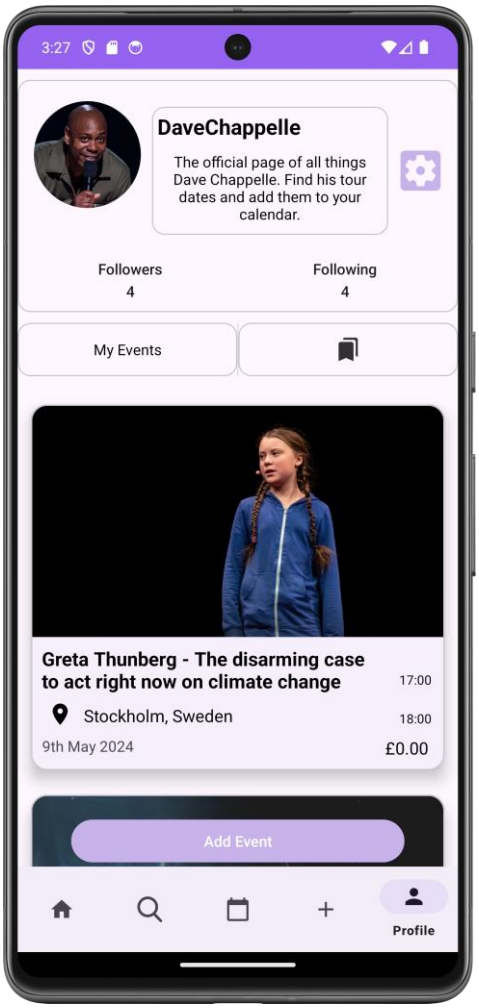


Figure 62 - Profile Screen (Saved Events)

4.12.3.1 Security

SavedEvents collection's security rules (Figure 63) are relatively simple, with users only being able to create a document where the id of the document is equal to their user id, and then only being able to access update or delete any documents if their user id matches the document id.

```
match /SavedEvents/{userId} {
  // Allow any authenticated user to create an event, and the document id(userID) must be the user's ic
  allow create: if request.auth != null && request.resource.id == request.auth.uid;

  // Allow anyone to read events
  allow read: if request.auth != null;

  // Allow only the user who created the event to update or delete it
  allow update: if request.auth.uid == resource.id;

  allow delete: if request.auth != null && request.auth.uid == resource.id;
}
```

Figure 63 - SavedEvents Security Rules

4.13 Settings Screen

The Settings Screen (Figure 64) allows for account editing, logging out and account deletion.

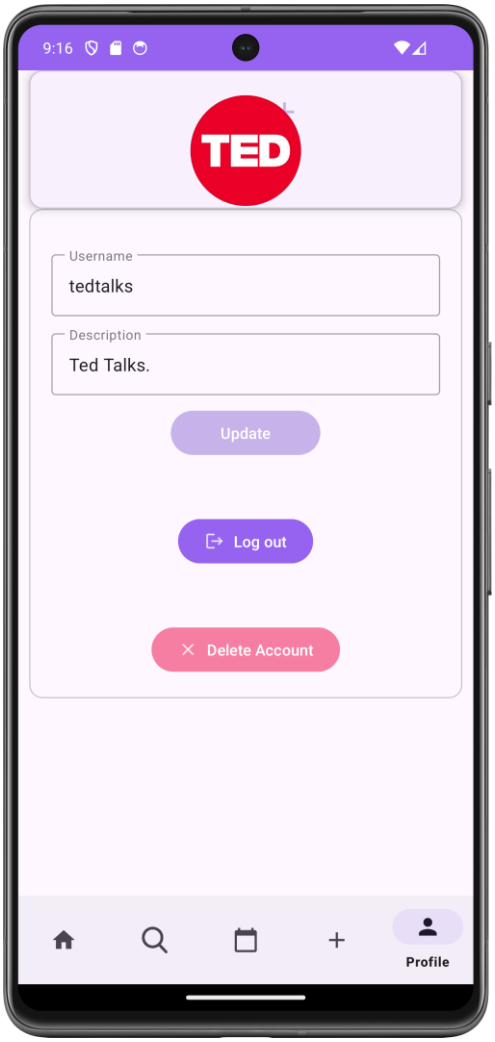


Figure 64 - Settings Screen

4.13.1.1 Account Editing

Users can alter their accounts username, description, and profile picture, this can then be confirmed by clicking the update button.

4.13.1.2 Log out button

Logging out is essential for users that which to switch between accounts and has a confirmation message to make sure the user wishes to sign out. This logs out the application from the signed in *FirebaseUser and* returns the user to the *SignupLoginFragment*.

4.13.1.3 Delete Account

The DPA (Data Protection Act, 2018) states that users must be able to delete their data from any holder of said data, upon request. Therefore, in-app account deletion is necessary. This button deletes the user's account details, authentication and any public events created by the account. The user must then re-enter their account details (Figure 65), to authorise with Firebase that the account is to be deleted.

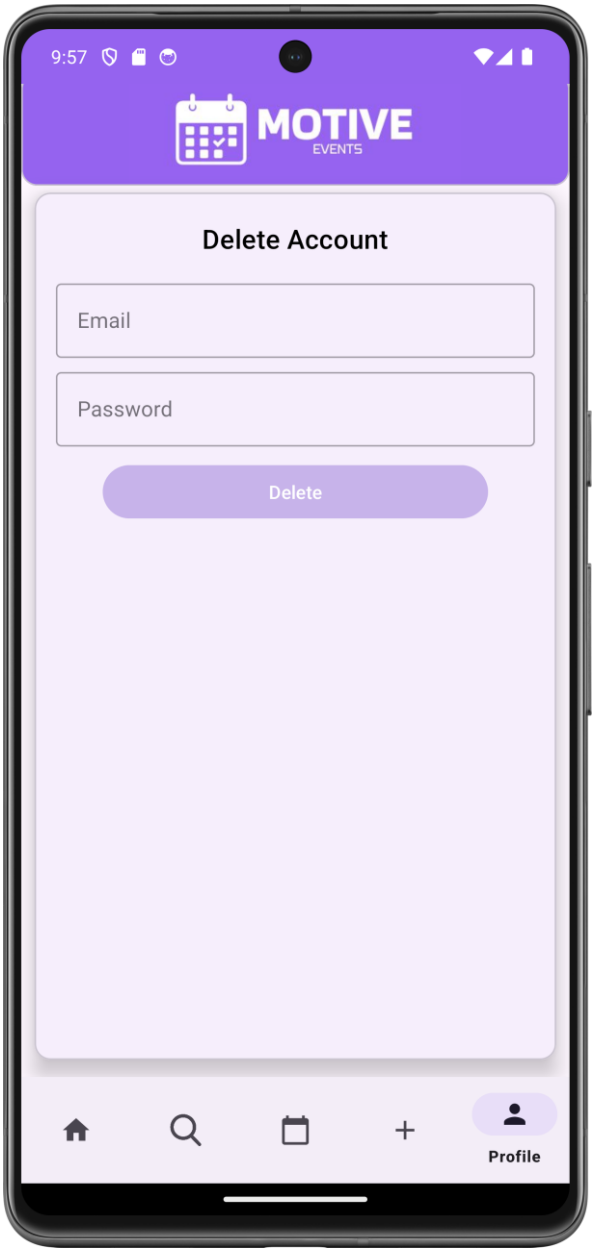


Figure 65 - Delete Account Confirmation

5. Testing

5.1 Functional Testing

Testing has been carried out for the functionality of the different features of the application of this project. This has been done by making sure the application adheres to the pre-defined requirements. This section will cover the outcome of these tests, going screen by screen. Each functional requirement will be tested and then graded on a pass/fail scale. All tests were carried out with a Google Pixel.

Each table contains columns, element, expected result, pass/fail and remarks. Element gives a short description of what is being tested on the screen. Expected result describes the full expected result in the form of “A user is able to/can” and what is expected of that element. Remarks are for any additional information, or for when a test fails, and describing what it failed, and how it was fixed.

5.1.1 Sign-up Screen

Table 12 - Sign-up Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Username Input	A user is able to enter their desired username, and is prompted if incorrectly formatted.	Pass	
Email input	A user is able to enter their email address, and if not a valid email address, then notified	Pass	
Password input	A user is able to enter their password	Pass	
Confirm password	A user is able to enter their password again, and if it matches, then it is valid, and if it doesn't then it displays that they don't match.	Pass	
Sign up button	A user is able to sign up to the application	Pass	
Check email button	A user is able to check their email verification, and if verified, directed to the MainActivity, and if not is notified of no verification.	FAIL	User was allowed to have no profile picture which resulted in failure of account creation, now profile picture is not required for account creation
No connection to Firebase Authentication	A user is notified of a connection error if no connection to Firebase	Pass	
Profile image upload	A user can upload a profile image either from their device or take a photo	FAIL	If no profile image was present, user upload failed.

5.1.2 Login Screen

Table 13 - Login Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Email input	A user is able to enter their email address, and if not a valid email address, then notified	Pass	
Password input	A user is able to enter their password, and if incorrect, is notified	Pass	
Login button	A user is able to login, if valid details are entered	Pass	
No connection to Firebase Authentication	A user is notified of a connection error if no connection to Firebase	Pass	

5.1.3 Home Screen

Table 14 - Home Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Discover tab	A user can click on the discover tab, it will be highlighted, and events will be shown from any user.	Pass	
Following tab	A user can click on the discover tab, it will be highlighted, and events will be shown from followed users.	Pass	
Event view	A user can click on an event and be taken to it's corresponding event page.	Pass	

5.1.4 Public Event View Screen

Table 15 - Public Event View Testing

Element	Expected Result	Pass/Fail	Remarks
Calendar Preview View	A user can see the calendar preview, where it shows the day preview to the event, to the day after the event.	Pass	
Calendar Preview attendance	A user can attend an event, and the calendar preview will be updated, showing the event is in the calendar.	Pass	
Calendar Preview inner functionality	A user can click on a day of the calendar preview and is taken to the day view of selected day, and if long held on a day, taken to the add private event page of said day.	Pass	
Saving events	A user can save an event public ally, and then be viewed in the saved events tab of their profile screen	Pass	
Attendee Preview	A user can view whether a user they follow is attending an event. Cannot be users which they do not follow	Pass	
Information displayed	A user can view the event's date, title, location, link	Pass	
Profile picture	A user can view the event creator's profile picture, and once clicked, is taken to the said profile's page.	Pass	
Clickable link	A user can click the link and be taken to the website linked.	Pass	
Clickable location	A user can click the location symbol and google maps will be opened with the location loaded.	Pass	

5.1.4.1 Own User's Event

There are some separate functionalities of when viewing an event, if the user is the creator of the event, so they have been recorded in a separate table.

Table 16 - Public Event View Testing (Own User's Event)

Element	Expected Result	Pass/Fail	Remarks
Edit button	A user is able to click edit, and is taken to the add/edit page with the event loaded.	Pass	

5.1.5 Search Screen

Table 17 - Search Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Text search	A user is able to enter some text, and returns a set of events which are found by Algolia, which can be found across all the attributes of the event.	Pass	
Advanced search – min and max price	A user is able to set the minimum and maximum price of their search, and no events outside of that range will be shown	Pass	
Advanced search – select date	A user is able to set a date for the search, only events from that date are shown	Pass	
Advanced search - genre	A user can set the search to a specific genre, then is only shown events of that genre, or if set to any, then shown all genres	Pass	
Advanced search - tags	A user can set the search to only include results containing up to 10 tags, if inputting 11 tags, should notify user of the error.	Pass	

5.1.6 Calendar Screen

Table 18 - Calendar Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Month View	A user can see a view of a month upon clicking on the calendar symbol	Pass	
Year View	A user is able to click the month and year header, and have the year preview appear	Pass	
Previous and Next buttons	A user can go to previous month/year using the previous and next button dependent on the current view.	Pass	
Click to day view	A user is able to click on a day in month view and is taken to the day view of that day.	Pass	
Long click to add edit public	A user can long click on a day to take them to the add edit page, with the date selected being the day they long clicked	Pass	
Year view month selection	A user is able to click on a month in the year view, and be taken to the month view of the month selected	Pass	

5.1.7 Day View Screen

Table 19 - Day View Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Add event button	A user is able to click the add event button, and is taken to the add private event screen, with the current date loaded.	Pass	
Delete event button	A user can delete an event by clicking the delete event button and confirming it's deletion.	Pass	
Clicking on an event	A user can click on an event, and then be taken to the edit private event page with the clicked event loaded.	FAIL	When a public event that had been saved had been clicked on, it did not parse the alert time correctly. Now added appropriate exception.
Day selection from week	A user can click on a day of the week and be taken to the day view of that day, with the day loaded being highlighted	Pass	

5.1.8 Add/Edit Private Event Screen

Table 20 – Add/Edit Private Event Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Title	A user is able to enter a title for their event, and if over 60 characters, an error is set in layout.	Pass	
Location	A user is able to enter a location for the event.	Pass	
Description	A user is able to enter a description for the event.	Pass	
Event Type	A user can set the event type from the spinner, and this event type displays the correct colour event in other views.	Pass	
Date selection	A user can click the select date button, and is presented with a date picker, which if confirmed, the date is set, and if not, then notify the user	FAIL	If no date or start and end times are selected, then a crash would occur. Fixed by adding a null pointer check.
Start time and end time	A user can set the start time and end time of an event, if the end time is before the start time, notify the user	Pass	

5.1.9 Add/Edit Public Event Screen

Table 21 - Add Public Event Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Title	A user is able to enter a title for their event, and if over 100 characters, an error is set in layout	Pass	
Location	A user is able to enter a location for the event.	Pass	
Description	A user is able to enter a description for the event.	Pass	
Link	A user is able to enter a link for the event.	Pass	
Price	A user can enter a price for event, can only be numbers.	Pass	
Tags	A user can enter up to 10 tags and a minimum of 1 tag, showing both error messages for either condition.	Pass	
Genre	A user can set a genre from the spinner	Pass	
Event Type	A user can set the event type from the spinner	Pass	
Date selection	A user can click the select date button, and is presented with a date picker, which if confirmed, the date is set, and if not, then notify the user.	Pass	
Start time and end time	A user can set the start time and end time of an event, if the end time is before the start time, notify the user	Pass	
Banner image upload	A user can upload a banner image for their event, either being from the camera, or from their device.	Pass	

5.1.9.1 Editing an event

Editing an event has some of the same functionalities but also some extra so it has been split into a separate table for functionality coverage.

Requirement for each element in table: Upon clicking edit on Public Event View, the previous value should be loaded into the view.

Table 22 - Edit Public Event Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Title	A user is able to edit a title for their event, and if over 100 characters, an error is set in layout	Pass	
Location	A user is able to edit a location for the event.	Pass	
Description	A user is able to edit a description for the event.	Pass	
Link	A user is able to edit a link for the event.	Pass	
Price	A user can edit a price for event, can only be numbers. Upon clicking edit the previous value will be loaded into the view.	Pass	
Tags	A user can edit up to 10 tags and a minimum of 1 tag, showing both error messages for either condition..	Pass	
Genre	A user can change the genre from the spinner.	Pass	
Event Type	A user can set the event type from the spinner.	Pass	
Date selection	A user can click the select date button, and is presented with a date picker, which if confirmed, the date is set, and if not, then notify the user.	Pass	
Start time and end time	A user can edit the start time and end time of an event, if the end time is before the start time, notify the user.	Pass	
Banner image upload	A user can edit a banner image for their event, either being from the camera, or from their device.	Pass	
Event Deletion	A user can delete an event by clicking the delete button.	Pass	

5.1.10 Profile Screen

Table 23 - Profile Screen Testing

Element	Expected Result	Pass/Fail	Remarks
My Events Tab	A user is able to click the My Events tab, and shown the public events created by the user.	Pass	
Saved Events	A user is able to click the saved events tab, and presented with any events that the profile user has saved.	Pass	
Followers	A user is able to click the follower's card and then is presented with a list of the followers of a user.	Pass	
Following	A user is able to click the following card and then is presented with a list of the followers of a user.	FAIL	No following shown unless an event had been created, or an event had been saved, and the saved events tab had been clicked.
Settings button	A user is able to click the settings button and is taken to the settings screen.	Pass	

Table 21 - Profile Testing

5.1.11 Settings Screen

Table 24 - Settings Screen Testing

Element	Expected Result	Pass/Fail	Remarks
Username change	A user is able to input a username to change to, if the username is taken, the user will be notified.	Pass	
Description change	A user is able to input a new description.	Pass	
Profile Picture update	A user is able to update their profile picture.	Pass	
Log out button	A user is able to log out, and when they restart the application, their user will no longer be signed in either.	Pass	
Delete account	A user is able to delete their account, and unable to log back in. All user details must be deleted from the database.	Pass	

5.2 User Interaction Testing

This section details and analyses the testing done for user experience/interaction.

The testing consists of 6 testers who completed 3 tasks and were asked to give feedback in the form of 6 quantitative measures:

- Learnability, “how easy to learn”.
- Efficiency (number of clicks), “How efficient”.
- Satisfiability, “How satisfied”,
- Usefulness, “How useful in real life”.
- Aesthetics, “How aesthetically pleasing”.
- Legibility, “How easy to read”.

They were then asked to give remarks throughout the testing, and these remarks were used to make changes to the user experience.

5.2.1 Quantitative Testing

The following section displays the test description and data.

Each graph represents a task, with the y-axis displaying the rating given for a measure out of 10 (starting at 5 as no score was below 5), the x-axis displaying the qualitative measures, and each bar colour representing an individual tester.

Task 1: “Create a user account, and then go to your calendar and create a private event.”

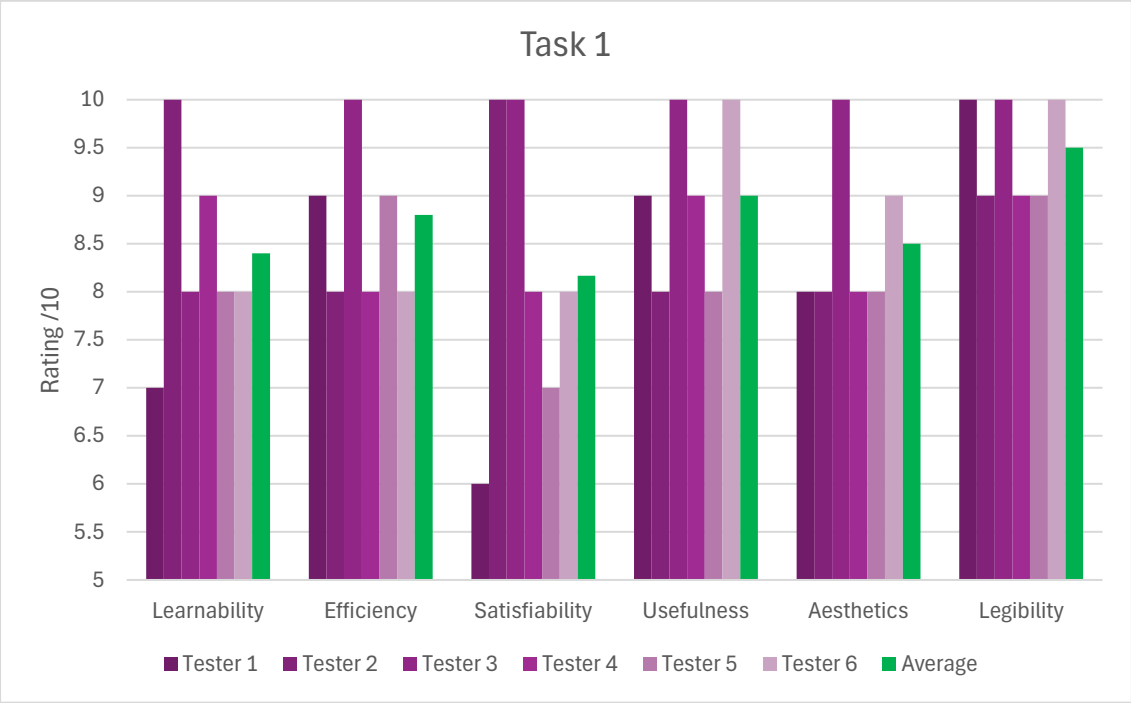


Figure 66 - Task 1 Results

Task 2: “Create a public event of your choice, then find created event using search, then edit your premade event by changing the event picture”.

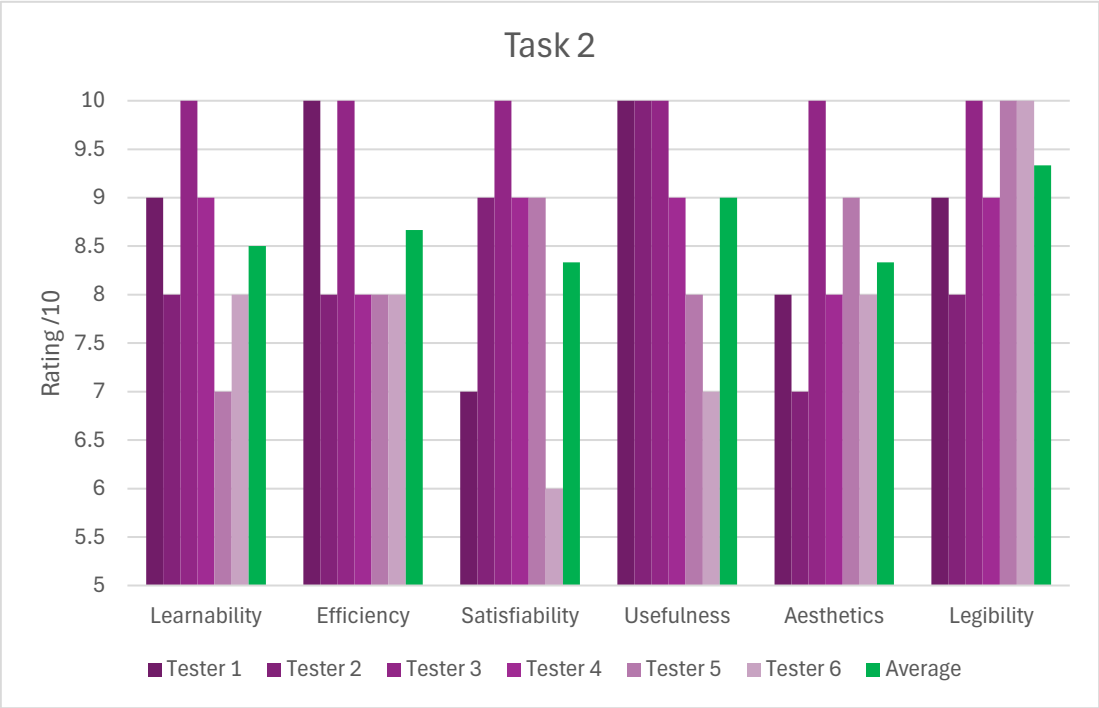


Figure 67 - Task 2 Results

Task 3: “Find an event on the discover page, then attend event by, adding event to private calendar and save event publicly. Start following the user, then go to calendar using the calendar preview and add an alert for the event.”

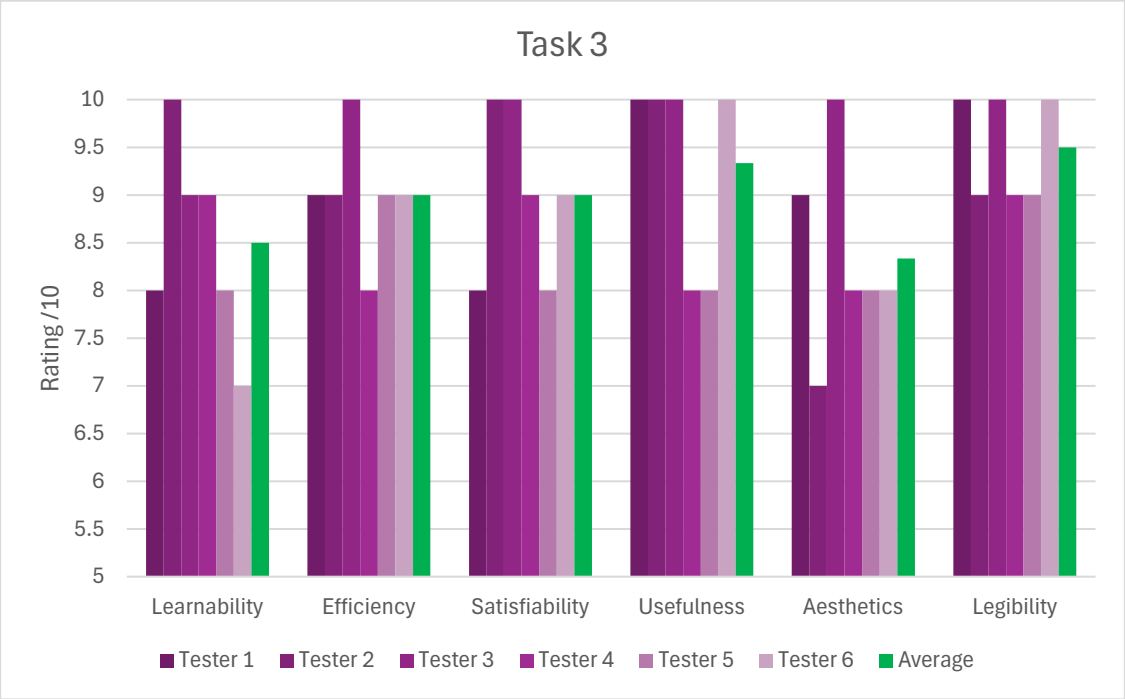


Figure 68 - Task 3 Results

5.2.2 Qualitative Testing

This section will go over some qualitative responses received, and general difficulties experienced during the tasks, and how the application has been changed to accommodate these difficulties.

5.2.2.1 Significant remarks

- Use of animations could improve user experience.
- Colour coding is marling a change, that instantiation is taking place (in reference to the colour coding of private events based on event type).
- Very aesthetically pleasing.
- Colour palette is fitting.
- Search is smooth and well-functioning.
- Some indication of advanced search would be good.
- Could use some icons for public event creation.

- Automatically updating upon attendance
- Productivity and organisation are expected (with the application).
- Calendar preview is interesting and innovative.

5.2.2.2 Changes

The criticisms from the remarks have been taken on board, and these changes have been made:

- the search screen, (upon the navigation tab being clicked), will now show a short text with some instructions, shown in Figure 33.
- Icons are now displayed next to inputs for the public event creation screen, shown in Figure 56.

5.2.2.3 Positive Feedback

There was a variety of positive feedback, and they have been compiled into this word cloud (Figure 69) as a visual representation.

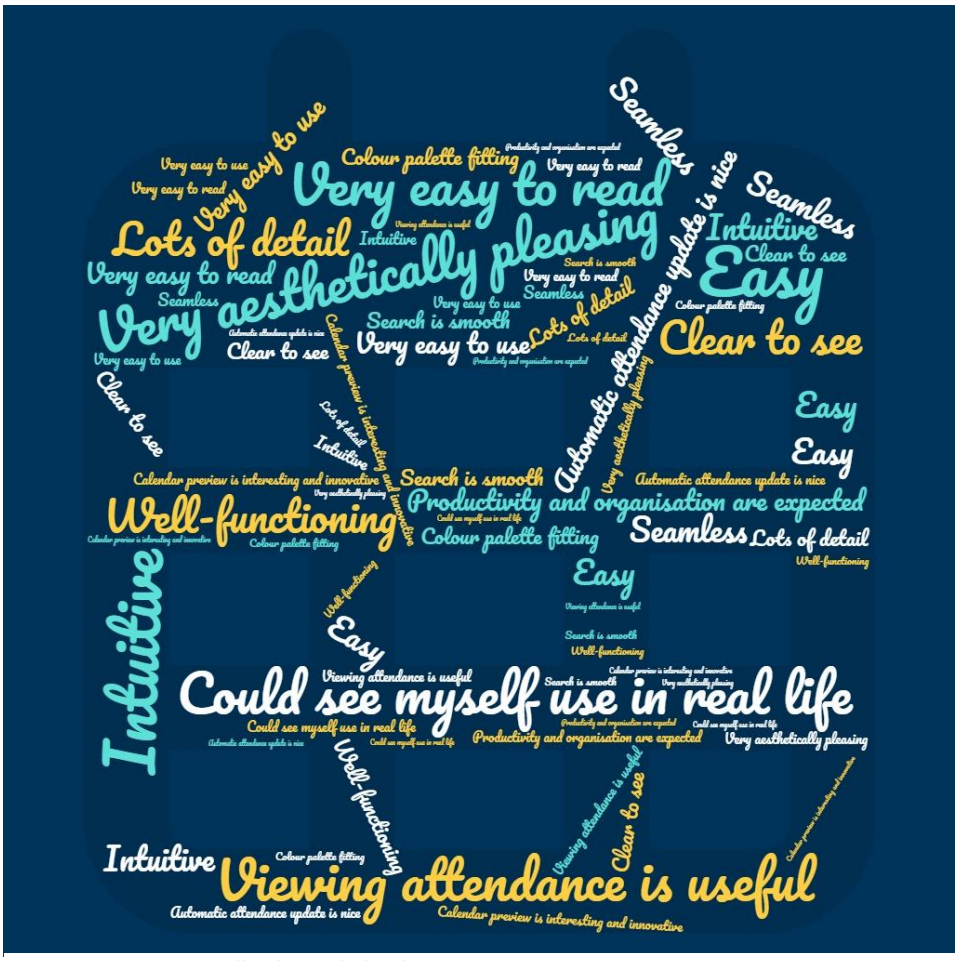


Figure 69 - Positive Feedback Word Cloud

5.2.2.4 Conclusion

Users found the application to be very legible, with the average across all tasks being >9/10. This shows that the application has clear text, with no layout issues apparent. On the other hand, satisfiability was recorded across the application as low. This could be due to the application's greater purpose is for the long term. This is as the users didn't have the opportunity or time to use over a prolonged period and integrate it within their routine. The lack of consistency could have led to the lack of overall satisfaction when testing in the small testing period.

To conclude, the test participants were asked how easy the application was to use, and whether they were likely to use if publicly released. These questions were asked based on the full application, rather than the previous, task-based feedback. The testers rated both questions out of 10. This graph follows the same schema as the previous task-based results.

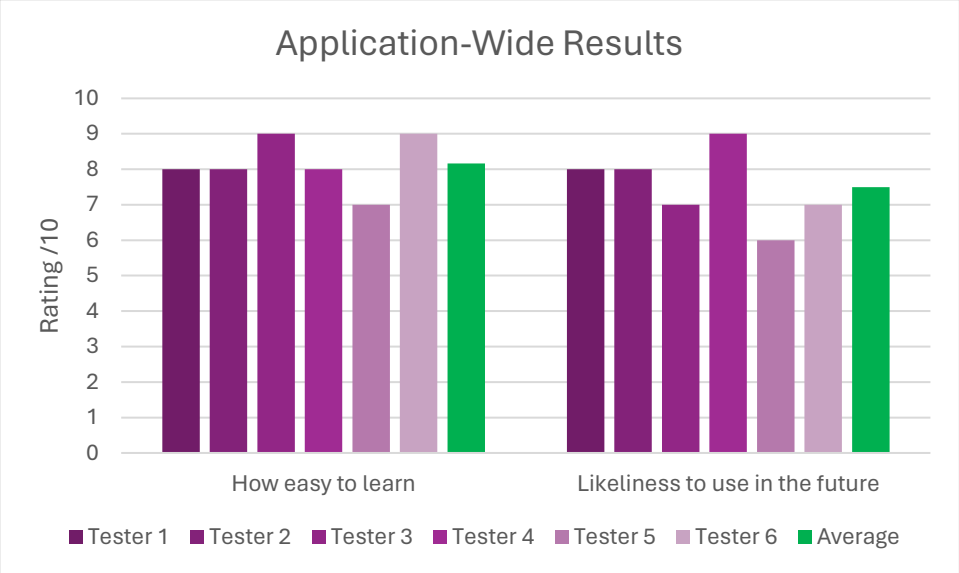


Figure 70 - Application-Wide Results

Figure 70 shows the results with learning the application appearing to be easy, with >8/10 average. The likelihood of using the application in the future was also positive with every user giving a 6 or above.

6. Conclusion

Overall, the application for this project has fulfilled the objectives laid out in the introduction of this report, with the innovation of the calendar preview within the public event view, creates a sense of control over the user's schedule. This allows the user to view the calendar around the date of the event they wish to attend and edit the calendar to remove items and make space for the new event they may attend. This calendar preview is also responsive, meaning it will update instantly upon attendance being set and this gives the user a great view of their schedule and allows them great control. This project's motivation of compiling the years of university study has been fulfilled with the diverse skills applied in the application and report, with database security, android development lifecycle and working with APIs, being key aspects of this project.

6.1 Learning curves

There was a lot to learn throughout this project, performing critical gap analysis, local database architecture, online database selection and security, working with external libraries, and deepening understanding of XML layouts and how to work with them in the Java code.

One of the greatest learning curves was the local database architecture, having to decide on the MVVM architecture, then understanding MVVM, and then applying that to this project's application. There was a foreseen great amount of learning in order to create the local database.

Another large learning curve faced was the handling of asynchronous data, where the data retrieved from the online database would have to be retrieved and transferred through methods via interfaces. This was done to prevent the sections of the code from executing without retrieving the data needed, by only allowing sections of code to be executed upon retrieval of the asynchronous data.

6.2 Challenges

This project's application came with many challenges which caused setbacks limited progress for a time. One of the key challenges in production were the security rules of the database.

Firstly, the needs of the rules must be decided, and once decided, then implemented. The implementation created the most disruption due to the lack of logging capabilities from Firebase. When the application returns an error for when the database doesn't allow the task to be done, it is returns "Missing Permissions or Permissions Denied", and when there are combinations of rules for one operation, for example the *allow update* rule for the *Followers* collection, then no exact rule will be returned as causing the failure. This then led to the consecutive removal and addition of rules to check which one is causing the fault, this, in combination with the update time for rule changes in Firebase, led to a slowing of development.

Secondly, understanding and determining the functionality of the application was found to be a challenge, where it would be capable of development within the project submission's timeframe. Developing the requirements from the decisions on functionality was hard and previous iterations of requirements lacked the innovation of the final set, with no calendar preview, which is now one of the most innovative features. Creating a clear set of requirements allowed for the development of the application to be smooth and without the determination of functionality.

6.3 Further Development

As time was limited on this project, while having to juggle other university work and life obligations, there is potential for further development on this project's application.

One of the ideas for development beyond this project's submission would be some form of collaborative calendar, where the user can view a combined calendar of a chosen group of their followers. This would show when the user group are available to attend an event together. On top of this, an algorithm could be developed which gathers the private events if each user in the user group, and then calculates when the users are all free, and then offers events within the time frames given by the algorithm. This would also require some form of in-app messaging to organise the collaborative event attendance.

6.4 Final Thoughts

Overall, I believe this project to have delivered its objectives and have successfully created an application and report that displays my computer science skills and innovation to create a product worthy of public distribution.

References

Abbas, M. (2024). mabbas007/TagsEditText. [online] GitHub. Available at: https://github.com/mabbas007/TagsEditText.

Algolia. (2023). *Algolia vs Elastic: Full text search in database*. [online] Available at: https://algolia.com/blog/engineering/full-text-search-in-your-database-algolia-versus-elasticsearch/

Bizzabo (2023). In-person B2B Conferences Trends in 2023: Report | Bizzabo. [online] Available at: https://welcome.bizzabo.com/in-person-conferences-state-report-2023

bumptech (2019). bumptech/glide. [online] GitHub. Available at: https://github.com/bumptech/glide.

Eventbrite (2018). Eventbrite. [online] Eventbrite. Available at: https://www.eventbrite.co.uk/.

Firebase. (n.d.). Cloud Storage for Firebase | Store and serve content with ease. [online] Available at: https://firebase.google.com/products/storage/.

Google (2019). Cloud Firestore | Firebase. [online] Firebase. Available at: https://firebase.google.com/docs/firestore.

Muhammad Nazrul Islam (2015). Exploring the intuitiveness of iconic, textual and icon with texts signs for designing user-intuitive web interfaces. [online] doi:https://doi.org/10.1109/iccitechn.2015.7488113.

Patel, D. (2022). 📸 Image Picker Library for Android. [online] GitHub. Available at: https://github.com/Dhaval2404/ImagePicker.

Silberschat et al. (2016). Database System Concepts - 6th edition. *Database System Concepts 6th edition.pdf*. [online] Google Docs. Available at: https://drive.google.com/file/d/13q8K7vhvQaavlgoO1eJpKQHKjaTBh8dP/view

Skiddle.com. (2019). Buy tickets for Gigs, Clubs and Festivals. Skiddle: discover great events. [online] Available at: https://www.skiddle.com/.

Statista. (2022). *skiddle brand profile UK 2022 | Statista*. [online] Available at: https://www.statista.com/forecasts/1335107/skiddle-event-tickets-brand-profile-in-the-uk.

Data Protection Act 2018, c. 12. Available at: https://www.legislation.gov.uk/ukpga/2018/12/contents/enacted.

Appendix

Figure 71 - User Testing Results

USER TESTING RESULTS

Task 1

Task Description = “Create a user account and add an event to your private calendar”

Tester 1

Question	Rating	Remarks
How easy to learn	7	Animations could be used to improve user experience. Clear to see, instructions were good.
How efficient	9	
How satisfying	6	
How useful	9	
How aesthetically pleasing	8	
How easy to read	10	

Tester 2

Question	Rating	Remarks
How easy to learn	10	Text confirmation would be nice
How efficient	8	
How satisfying	10	
How useful	8	
How aesthetically pleasing	8	
How easy to read	9	

Tester 3

Question	Rating	Remarks
How easy to learn	8	
How efficient	10	
How satisfying	10	
How aesthetically pleasing	10	
How enjoyable	10	
How easy to read	10	

Tester 4

Question	Rating	Remarks
How easy to learn	9	Colour coding is marking a change, and instantiation taking place. A process has happened.
How efficient	8	
How satisfying	8	
How useful	9	
How aesthetically pleasing	8	
How easy to read	9	

Tester 5

Question	Rating	Remarks
How easy to learn	8	
How efficient	9	
How satisfying	7	
How useful	8	
How aesthetically pleasing	8	
How easy to read	9	

Tester 6

Question	Rating	Remarks
How easy to learn	8	Very aesthetically pleasing. Colour palette fitting.
How efficient	8	
How satisfying	8	
How aesthetically pleasing	10	
How enjoyable	9	
How easy to read	10	

Task 2

Task Description – “Create a public event of your choice, then find created event using search, then edit your premade event by changing the event picture”.

Tester 1

Question	Rating	Remarks
How easy to learn	9	Maps for location would be nice. Search is smooth and well functioning.
How efficient	10	
How satisfying	7	
How aesthetically pleasing	10	
How enjoyable	8	
How easy to read	9	

Tester 2

Question	Rating	Remarks
How easy to learn	8	Seamless, easy, intuitive
How efficient	8	
How satisfying	9	
How useful	10	
How enjoyable	7	

How difficult to read	8	

Tester 3

Question	Rating	Remarks
How easy to learn	10	
How efficient	10	
How satisfying	10	
How useful	10	
How enjoyable	10	
How difficult to read	10	

Tester 4

Question	Rating	Remarks
How easy to learn	9	Some indication of advanced search would be good.
How efficient	8	
How satisfying	9	
How useful	9	
How aesthetically pleasing	8	
How easy to read	9	

Tester 5

Question	Rating	Remarks
How easy to learn	7	Very easy to read. Could use some icons for the public event creation.
How efficient	8	
How satisfying	9	
How useful	8	
How aesthetically pleasing	9	
How easy to read	10	

Tester 6

Question	Rating	Remarks
How easy to learn	8	
How efficient	8	
How satisfying	6	
How useful	7	
How aesthetically pleasing	8	
How easy to read	10	

Task 3

Task Description – “Find an event on the discover page, then attend event by, adding event to private calendar and save event publicly. Start following the user, then go to calendar using the calendar preview and add an alert for the event.

Tester 1

Question	Rating	Remarks
How easy to learn	8	Lots of detail, automatically updating upon attendance is nice. Some indication of profile username or profile image clickability.
How efficient	9	
How satisfying	8	
How useful	10	
How aesthetically pleasing	9	
How easy to read	10	

Tester 2

Question	Rating	Remarks
How easy to learn	10	Got most needed features. Productivity and organisation are expected.
How efficient	9	
How satisfying	10	
How useful	10	
How aesthetically pleasing	7	
How easy to read	9	

Tester 3

Question	Rating	Remarks
How easy to learn	9	Very easy to use and could see themselves use it in real life.
How efficient	10	
How satisfying	10	
How useful	10	
How aesthetically pleasing	10	
How easy to read	10	

Tester 4

Question	Rating	Remarks
How easy to learn	9	Pop up for alert upon attendance or maybe one time pop-up which is changeable in settings. Viewing who is attending is useful.
How efficient	8	
How satisfying	9	
How useful	8	
How aesthetically pleasing	8	
How easy to read	9	

Tester 5

Question	Rating	Remarks
How easy to learn	8	
How efficient	9	
How satisfying	8	
How useful	8	
How aesthetically pleasing	8	
How easy to read	9	

Tester 6

Question	Rating	Remarks
How easy to learn	7	Calendar preview is interesting and innovative.
How efficient	9	
How satisfying	9	
How useful	9	
How aesthetically pleasing	8	
How easy to read	10	

```
public static class InsertEventAsyncTask extends AsyncTask<PrivateEvent, Void, Void> {
    2 usages
    private EventDao eventDao;

    1 usage
    private InsertEventAsyncTask(EventDao eventDao) { this.eventDao = eventDao; }
    @Override
    protected Void doInBackground(PrivateEvent... events) {
        eventDao.insert(events[0]);
        return null;
    }

}

1 usage
private static class UpdateEventAsyncTask extends AsyncTask<PrivateEvent, Void, Void>{
    2 usages
    private EventDao eventDao;

    @Override
    protected Void doInBackground(PrivateEvent... events) {
        eventDao.update(events[0]);
        return null;
    }

    1 usage
    private UpdateEventAsyncTask(EventDao eventDao) { this.eventDao =eventDao; }
}

1 usage
private static class DeleteEventAsyncTask extends AsyncTask<PrivateEvent, Void, Void>{
    2 usages
    private EventDao eventDao;
    @Override
    protected Void doInBackground(PrivateEvent... events) {
        eventDao.delete(events[0]);
        return null;
    }

    1 usage
    private DeleteEventAsyncTask(EventDao eventDao) { this.eventDao = eventDao; }
}
```

Figure 72 - EventRepository 2